



P P SAVANI
UNIVERSITY

OPERATING SYSTEM

SEIT2230



P P SAVANI
UNIVERSITY

S1_Module_1

Introduction



P P SAVANI
UNIVERSITY

Syllabus

Module No.	Content	Hours	Weightage in %
1.	Introduction to Operating System Basics of Operating System: Definition, Types of Operating System, Operating System Structure, Operating System Services, Concept of Virtualization.	03	07



What is Operating System (OS)?

- A Computer consist of various hardwares such as



Processor



RAM



Keyboard & Mouse



Hard Disk



Monitor



Printer

Who manages (controls) these hardwares???

Operating System

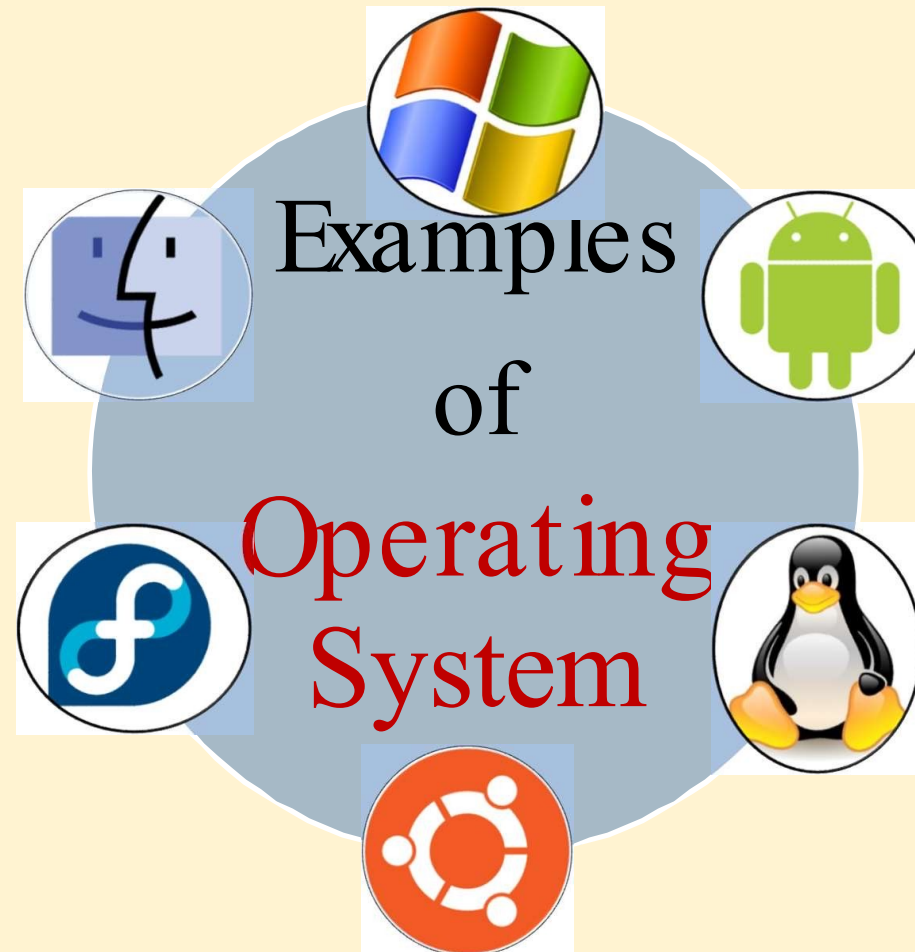


Definition of Operating System

- An Operating System (OS) is a collection of software that
 - manages hardware resources
 - provides various service to the users



Examples of Operating System



Operating system is like Government



P P SAVANI
UNIVERSITY

- Operating system provide the means for proper use of system resources.
- Government also provides facilities to the peoples like providing job opportunities, making new plans for facilities of peoples, solve problems of peoples.
- Like a government, OS performs no useful work itself but it provide an environment to use the computer system. So, operating system is similar to government.



Goals of an Operating System

- **Convenience:** Simplify the execution of user programs and make solving user problems easier
- **Efficiency:** Use computer hardware efficiently
 - Allow sharing of hardware and software resources.
- Make application software portable and versatile
- Provide isolation, security and protection among user programs
- Improve overall system reliability



Why should I study Operating Systems?

- Need to understand interaction between the hardware and applications
 - New applications, new hardware
- Need to understand basic principles in the design of computer systems
 - efficient resource management, security, flexibility
- Increasing need for specialized operating systems
 - e.g. embedded operating systems for devices - cell phones, sensors and controllers
 - real-time operating systems – vehicles, aircraft control, multimedia services



History of Operating Systems

Generation	Year	Electronic devices used	Types of OS and devices
First	1945 – 55	Vacuum tubes	Plug boards
Second	1955 – 1965	Transistors	Batch system
Third	1965 – 1980	Integrated Circuit (IC)	Multiprogramming
Fourth	Since 1980	Large scale integration	PC



P P SAVANI
UNIVERSITY

History of OS (First generation)

First generation (1945-1955)

- Vacuum tubes and plug-boards are used in these systems.



Vacuum tubes



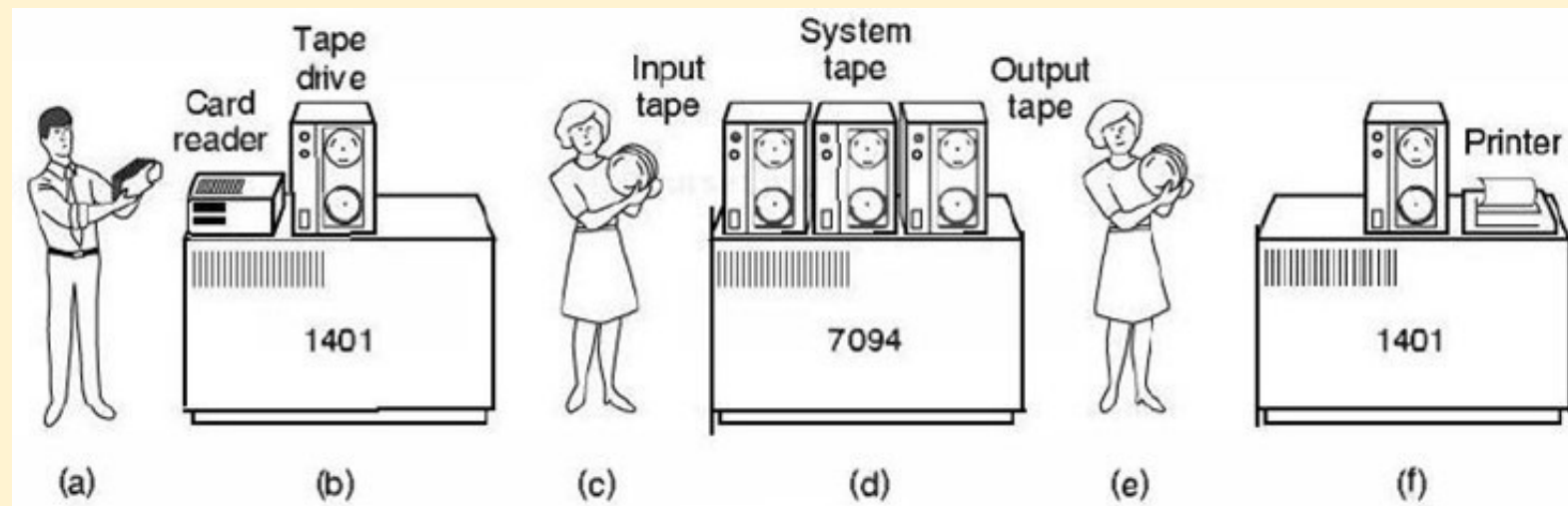
Plug board



History of OS (Second generation)

❖ Second generation (1955-1965)

- Transistors are used in these systems
- The machine that are produced are called mainframes.
- Batch systems was used for processing.





History of OS (Third generation)

❖ Third generation (1965-1980)

- Integrated circuits (IC's) are used in place of transistors in these computers.
- It provides **multiprogramming** (the ability to have several programs in memory at once, each in its own memory partition).





History of OS (Fourth generation)

- ❖ Fourth generation (1980-present)
 - Personal Computers
 - LSI (**Large Scale Integration**) circuits, chips containing thousands of transistors are used in these systems.





Early Systems - Bare Machine (1950s)

Hardware – *expensive* ; Human – *cheap*

- Structure
 - Large machines run from console
 - Single user system
 - Programmer/User as operator
 - Paper tape or punched cards
- Early software
 - Assemblers, compilers, linkers, loaders, device drivers, libraries of common subroutines.
- Inefficient use of expensive resources
 - Low CPU utilization, high setup time.



From John Ousterhout slides

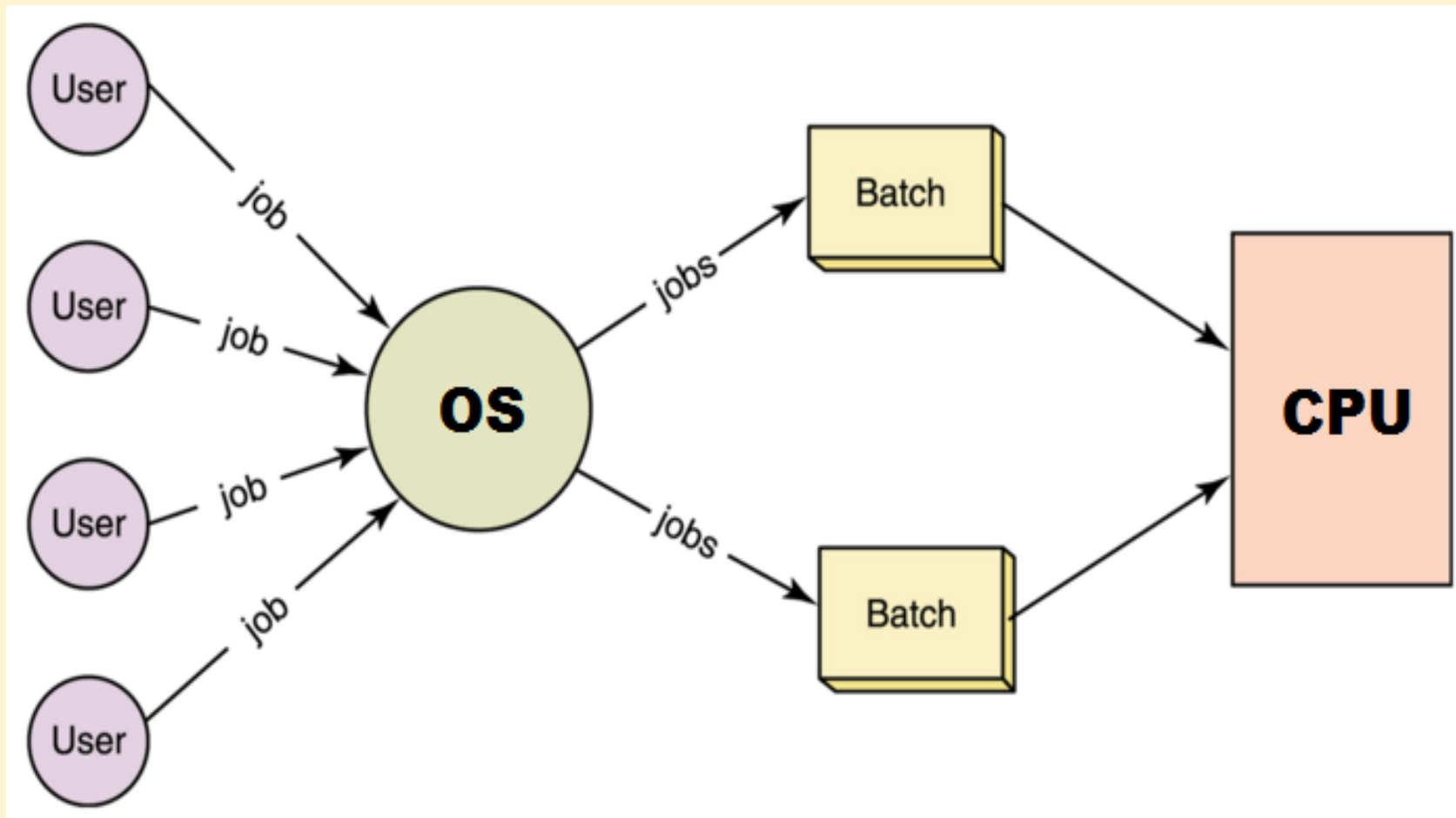


Simple Batch Systems (1960's)

- Batch operating system is one where programs and data are collected together in a batch before processing starts.
- A job is predefined sequence of commands, programs and data that are combined in to a single unit called job.
- Reduce setup time by batching jobs with similar requirements.
- Early systems - one job at a time
- Programmers would submit their programs to operator.
 - And operator would sort programs into batches with similar requirements.



Simple Batch Systems





Simple Batch Systems

- **Advantages**

- Move much of the work of the operator to the computer.
- Increased performance since it was possible for job to start as soon as the previous job finished.

- **Problems**

- Turn around time can be large from user standpoint.
- Low CPU utilization
 - I/O and CPU could not overlap; slow mechanical devices.
- Lack of interaction between the user and the job while job is executing



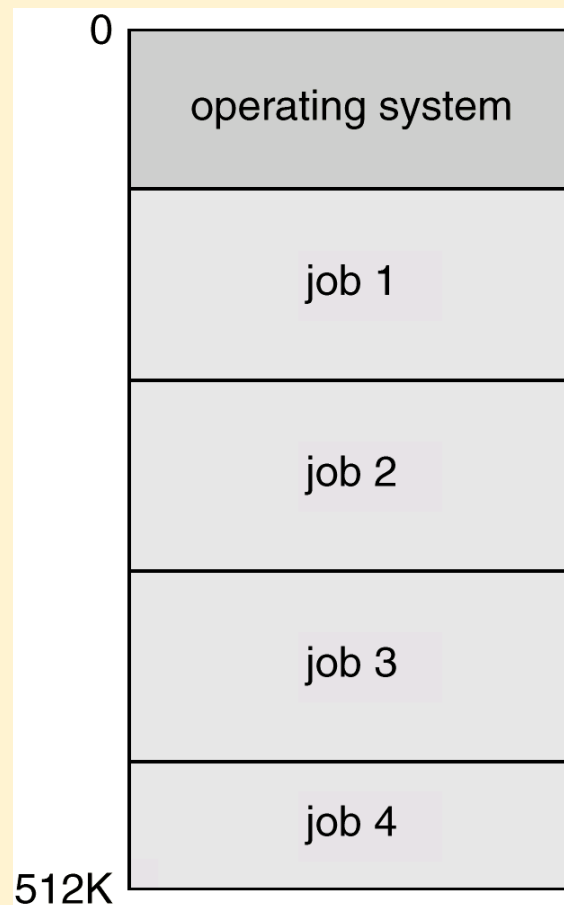
Simple Batch Systems

- Batch Systems are appropriate for Executing large jobs that need little interaction.



Multiprogramming Operating System

Several jobs are kept in main memory at the same time, and the CPU is multiplexed among them.





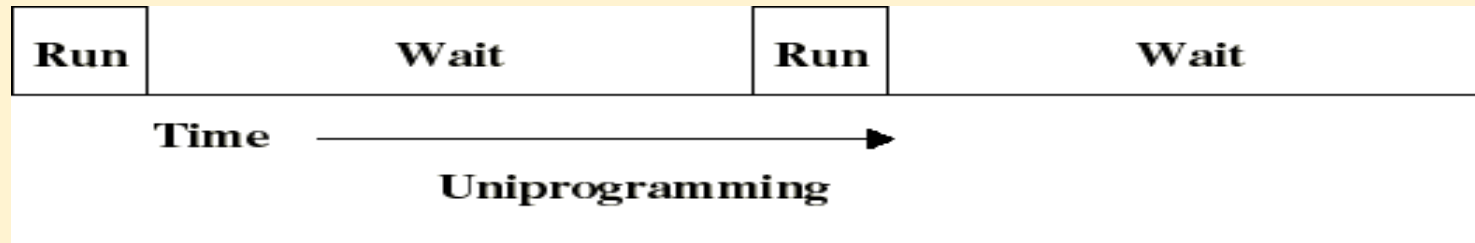
Multiprogramming OS

- Multiprogramming increases CPU utilization
- Multiple jobs are loaded into main memory and one is selected from pool for execution by CPU
- If at some point program in progress requires service of a peripheral device, the control of CPU is given to next job in the pool
- In this way CPU is always executing some program instead of waiting.

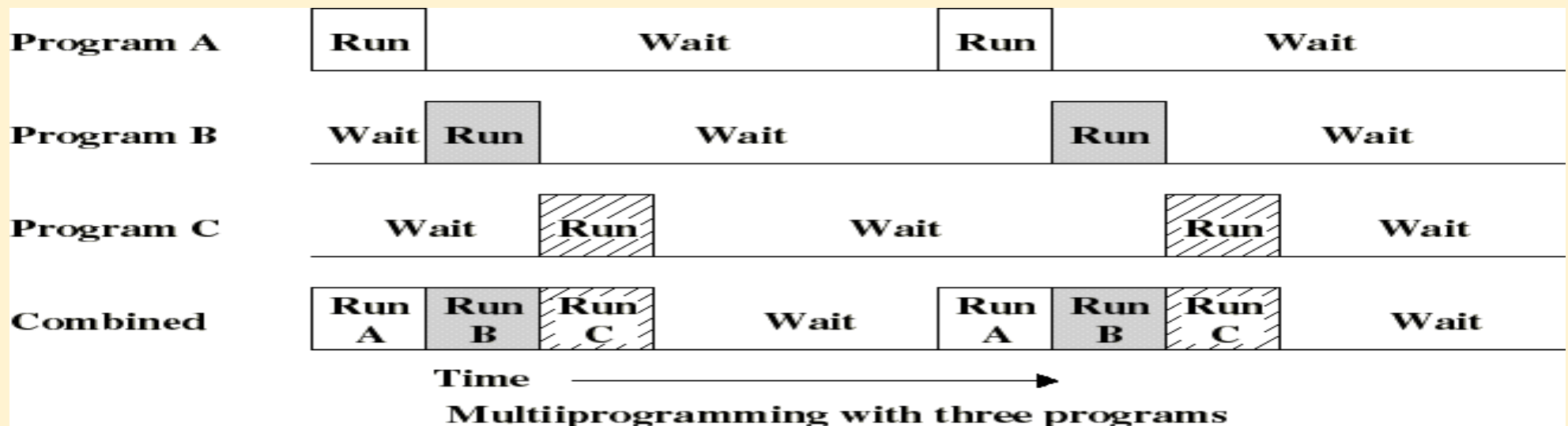
CPU usage is poor when only one program is present in memory



P P SAVANI
UNIVERSITY



- If memory can hold several programs, then CPU can switch to another one whenever a program is awaiting for an I/O to complete. This is multitasking (multiprogramming).





Example: 3 jobs are submitted

	JOB1	JOB2	JOB3
Type of job	Heavy compute	Heavy I/O	Heavy I/O
Duration	5 min.	15 min.	10 min.
Memory req.	50K	100 K	80 K
Need disk?	No	No	Yes
Need terminal	No	Yes	No
Need printer?	No	No	Yes



Advantages of Multiprogramming

	Uniprogramming	Multiprogramming
Processor use	17%	33%
Memory use	33%	67%
Disk use	33%	67%
Printer use	33%	67%
Elapsed time	30 min.	15 min.
Throughput rate	6 jobs/hr	12 jobs/hr
Mean response time	18 min.	10 min.



Multiprogramming

- **Advantages**
 - 1. High CPU utilization.
 - 2. It appears that many programs are allotted CPU almost simultaneously.
- **Disadvantages**
 - 1. CPU scheduling is requires.
 - 2. To accommodate many jobs in memory, memory management is required.
 - Multiprogramming does not support interaction with users



Time Sharing system (TSS)

- Multiprogramming does not support interaction with users
- TSS is logical extension of multiprogramming system
- TSS extends multiprogramming to handle multiple interactive jobs
- Processor's time is shared among multiple users
- Multiple users simultaneously access the system through terminals



P P SAVANI
UNIVERSITY

Time Sharing Operating System





Time-Sharing Systems–Interactive Computing

- The CPU is multiplexed among several jobs that are kept in memory and on disk (the CPU is allocated to a job only if the job is in memory).
- when the operating system finishes the execution of one command, it seeks the next “control statement” not from a card reader, but rather from the user’s keyboard.
- On-line system must be available for users to access data and code.



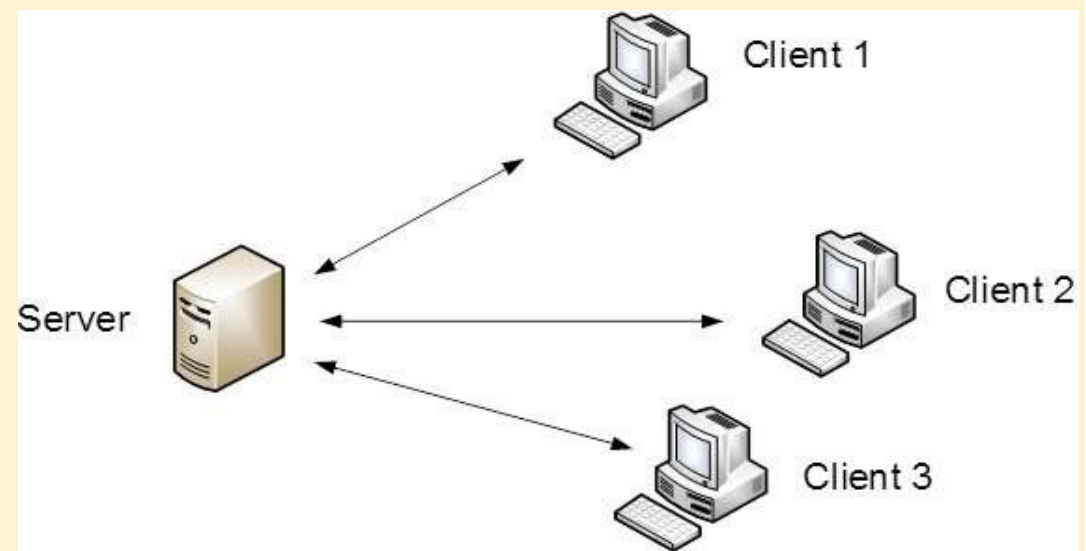
Time Sharing system (TSS)

- A time-shared operating system uses **CPU scheduling** and **multiprogramming** to provide each user with a small portion of a time-shared computer.
- Since each action or command in a time-shared system tends to be short, only a little CPU time is needed for each user.
- As the system switches rapidly from one user to the next,
 - each user is given the impression that she has her own computer, whereas actually one computer is being shared among many users.



Server operating systems

- This OS runs on servers which are very large PC, workstations or even mainframes.
- They serve multiple users at once over a network and allow the users to share hardware & software resources.
- It provides print services, file service or web service.
- It handles the incoming requests from clients.
- Examples: Solaris, FreeBSD, and Linux and Windows Server 200x.

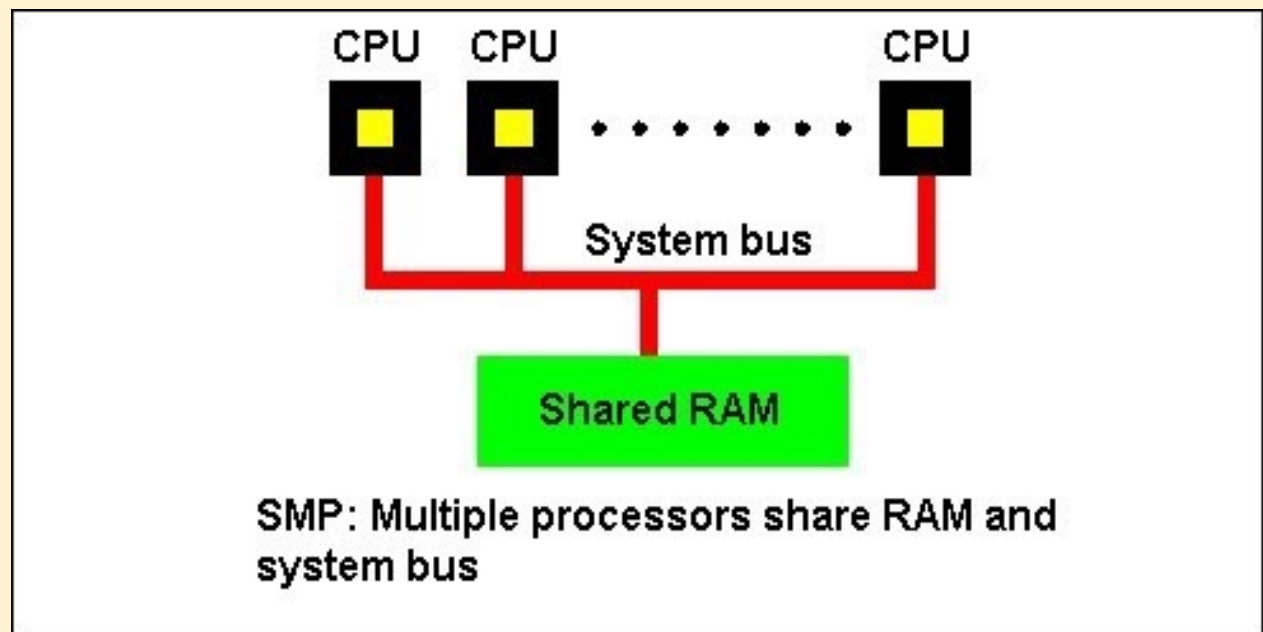


Multiprocessor operating systems (Parallel System)



P P SAVANI
UNIVERSITY

- A computer system consist two or more CPUs is called multiprocessor.
- It is also called parallelcomputers, multicomputer or multiprocessor.
- They need special OS or some variations on server OS with special features for communication, connectivity and consistency.
- **Examples:** Windows and Linux.





Parallel System

- **Increased *throughput*:**

- *By increasing number of processors we hope to get more work done in shorter period of time.*

- **Economical**

- *The processors can share peripherals, cabinets and power supplies.*
- *If several programs are to operate on same data*

- **Increased reliability**

- The failure of one processor will not halt the system, if functions can be distributed properly among several processors.



Real Time System

- A **real-time operating system (RTOS)** is an operating system that guarantees a certain capability within a specified time constraint.
- Hard real-time system, if the system fails to meet the deadline even once the system is considered to have Failed. Ex- Defense applications, nuclear system etc.
- Soft real-time system, even if the system fails to meet the deadline, possibly more than once (i.e. for multiple requests), the system is not considered to have failed. But, in this case the results of the requests are not worthless. Ex- audio-video streaming etc.



Distributed System

- Distribute the computation among several physical processors.
- *Loosely coupled system* – each processor has its own local memory; processors communicate with one another through various communications lines, such as high-speed buses or telephone lines.
- **Advantages of distributed systems.**
 - ✓ Resources Sharing
 - ✓ Computation speed up – load sharing
 - ✓ Reliability
 - ✓ Communications



Embedded operating systems

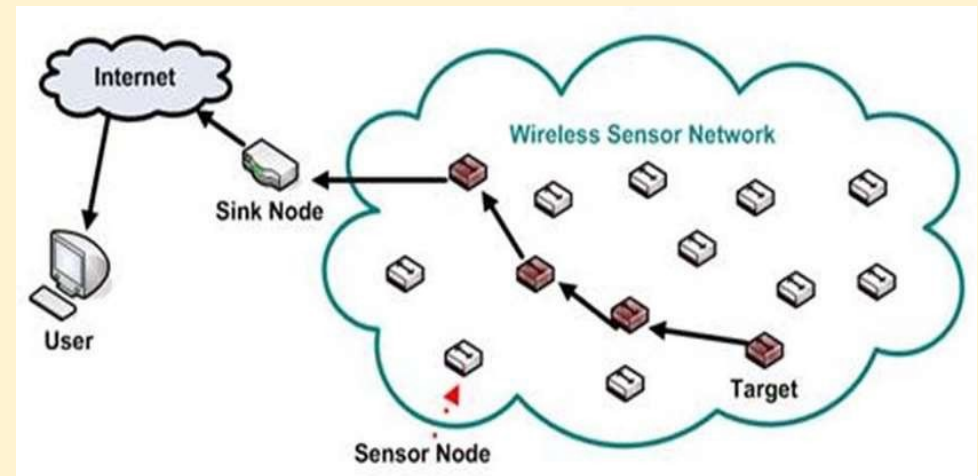
- This OS is installed in ATMs, printers, calculators and washing machine.
- It runs on the computer that control devices.
- It neither allow to download new software nor accept user installed software. So there is no need for protection.
- Examples: QNX, VxWorks.





Sensor node operating systems

- Network of tiny sensor nodes are being developed for numerous purpose.
- Each nodes are tiny computers with a CPU, RAM, ROM and one or more environmental sensors.
- The OS installed in these nodes are sensor node OS.
- They communicate with each other and with base station using wireless communication.
- These sensor network are used to protect area of building, detect fires in forest, measure temperature.
- Examples: TinyOS.





Smart card operating systems

- Smallest OS run on smart cards which are credit card sized devices containing CPU chip.
- These OS are installed on electronic payments cards such as debit card, credit card etc.
- They have limited processing power.
- Some smart cards are Java oriented. ROM on smart card holds an interpreter for the JVM – small program.



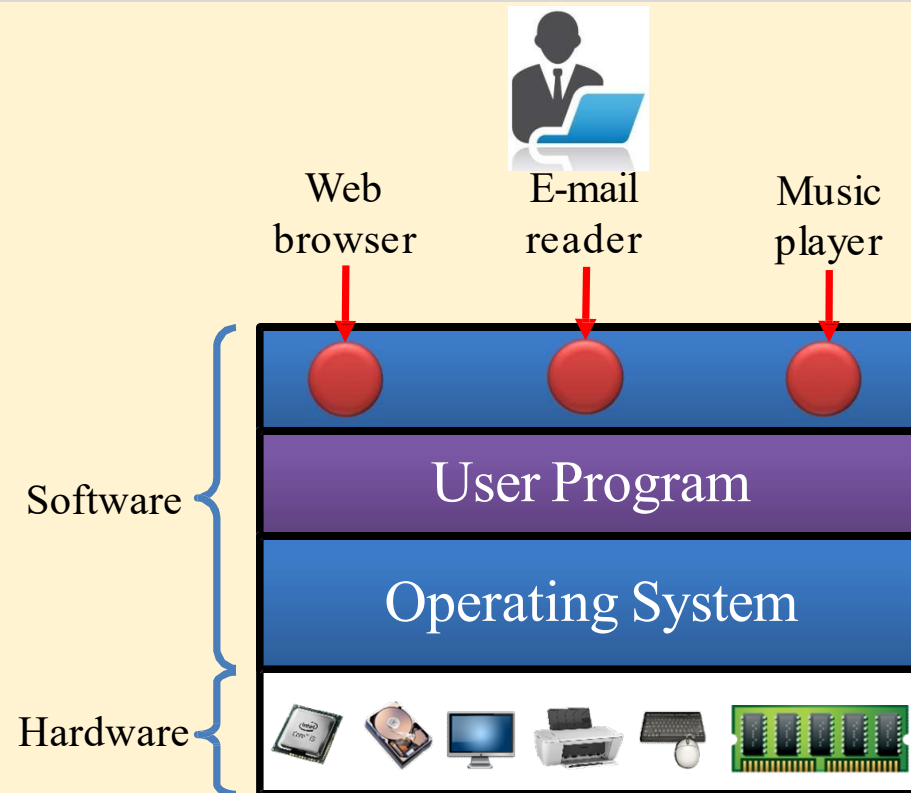


Personal Computer system

- ***Personal computers*** – computer system dedicated to a single user.
- **I/O devices** – keyboards, mice, display screens, small printers.
- User convenience and responsiveness.
- Can adopt technology developed for larger operating system' often individuals have sole use of computer and do not need advanced CPU utilization of protection features.



Where OS lies? Structure



- OS lies between hardware and user program.
- It acts as an interface between the user and the hardware.

Modes of operation of computer



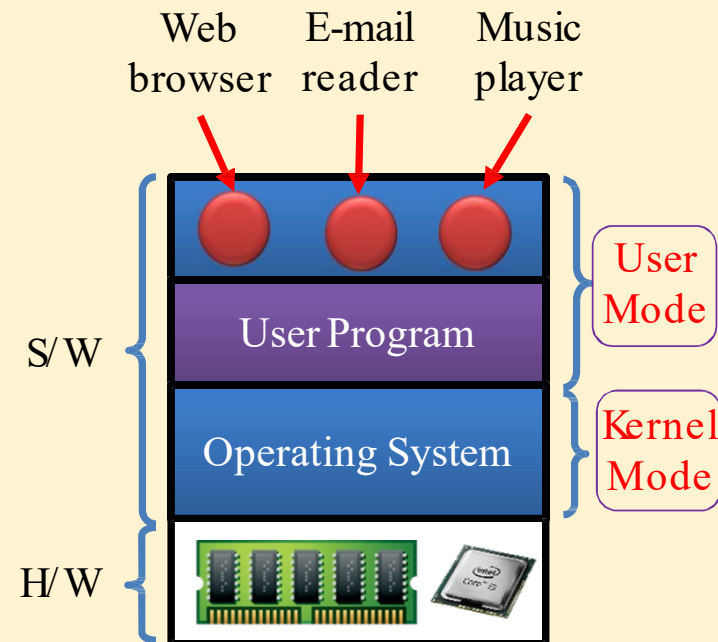
P P SAVANI
UNIVERSITY

1. Kernel Mode

- has complete access to all the hardware
- can execute any instruction that a machine is capable of executing
- has high privileged (rights)

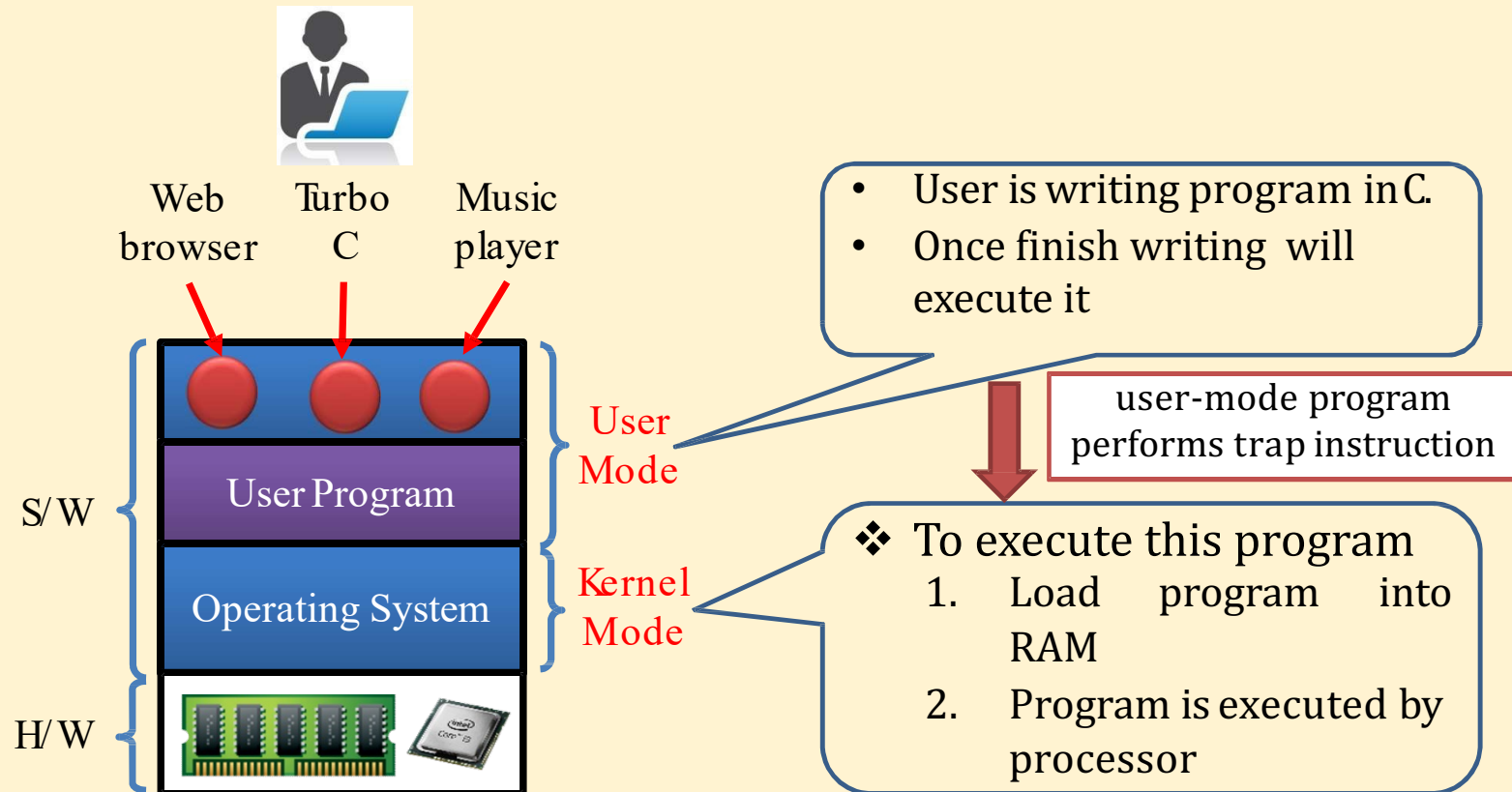
2. User Mode

- can execute only subset (few) of the machine instructions
- Less privileged (rights)





Why and How switch occur





Example – OS Involvement

Assume we are using MS-Paint on Windows - when do we need to access the OS?

- Loading the application / terminating the application
- Memory allocation / management (e.g., paging)
- Access to IO devices – keyboard, mouse, printer, monitor
- CPU allocation
- Copy / Paste (inter-process communication)



OS Services / Task of OS

Definition of Operating System

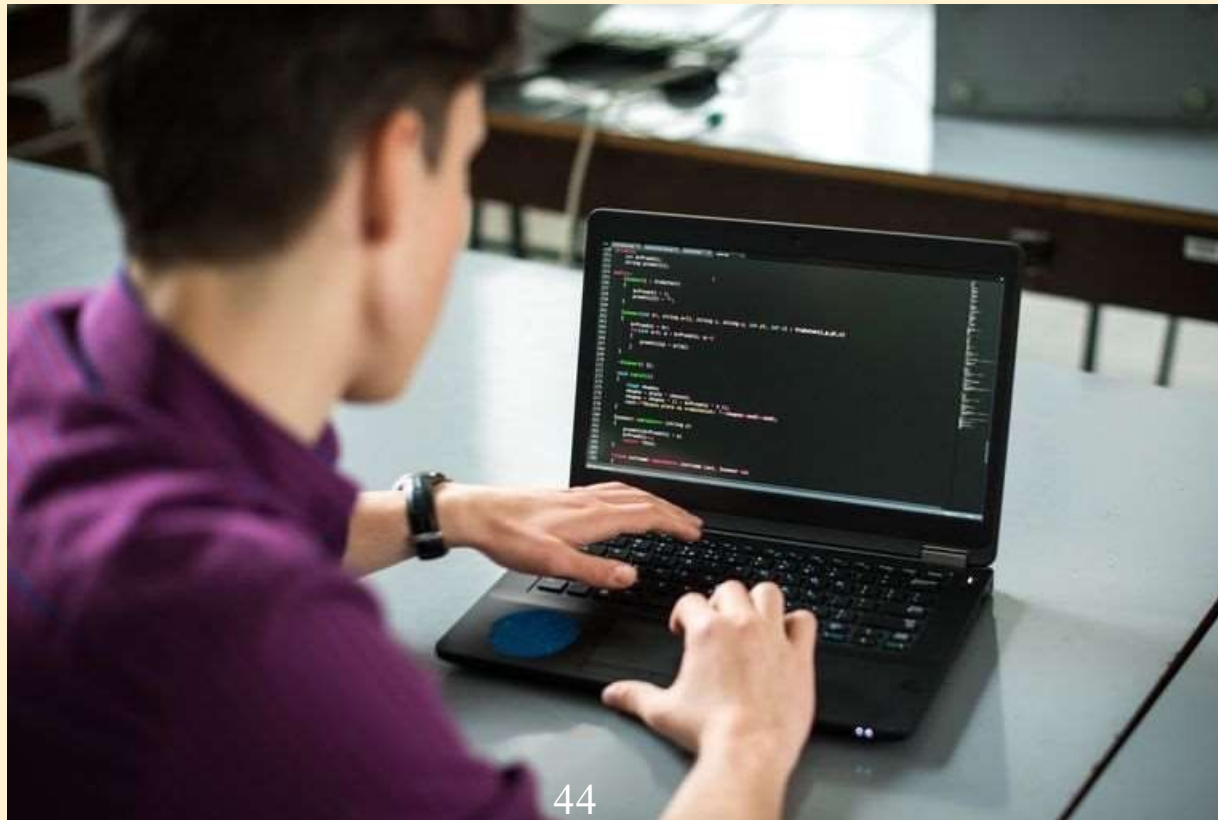
- ❖ An Operating System (OS) is a collection of software that
 - manages hardware resources
 - provides various service to the users



OS Services / Task of OS

1. Program development

- It provides editors and debuggers to assist (help) the programmer in creating programs.

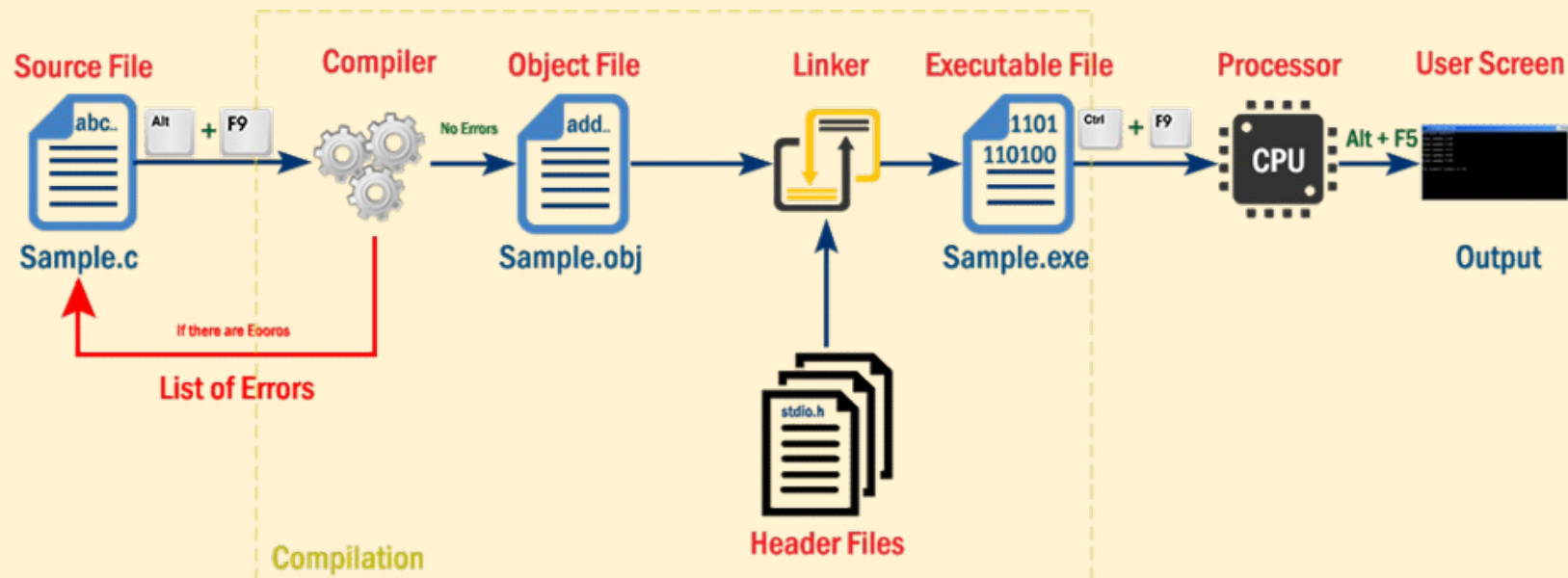




OS Services / Task of OS

2. Program execution

- Following tasks need to be performed to execute a program:
 - Instructions and data must be loaded into main memory.
 - I/O devices and files must be initialized.
- The OS handles all these duties for the user.





OS Services / Task of OS

3. Access to I/O devices (Resource allocation)

- A running program may require I/O, which may involve file or an I/O device.
- For efficiency and protection, users cannot control I/O devices directly.
- Therefore, the OS controls these I/O devices and provides to program as per requirement.

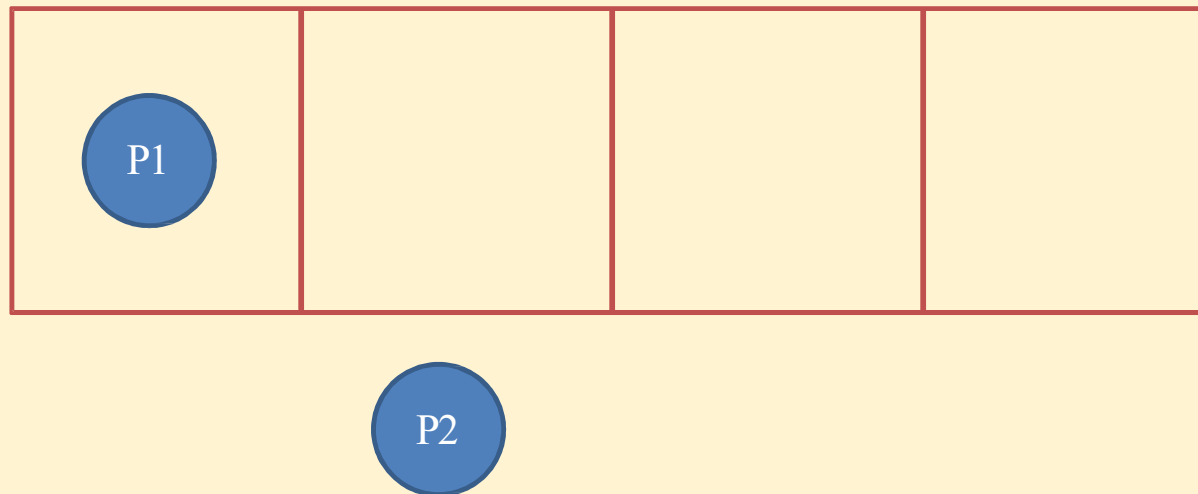




OS Services / Task of OS

4. Memory management

- OS manages memory hierarchy.
- OS keeps the track of which part of memory area in use and free memory.
- It allocates memory to program when they need it.
- It de-allocate the memory when the program finish execution.

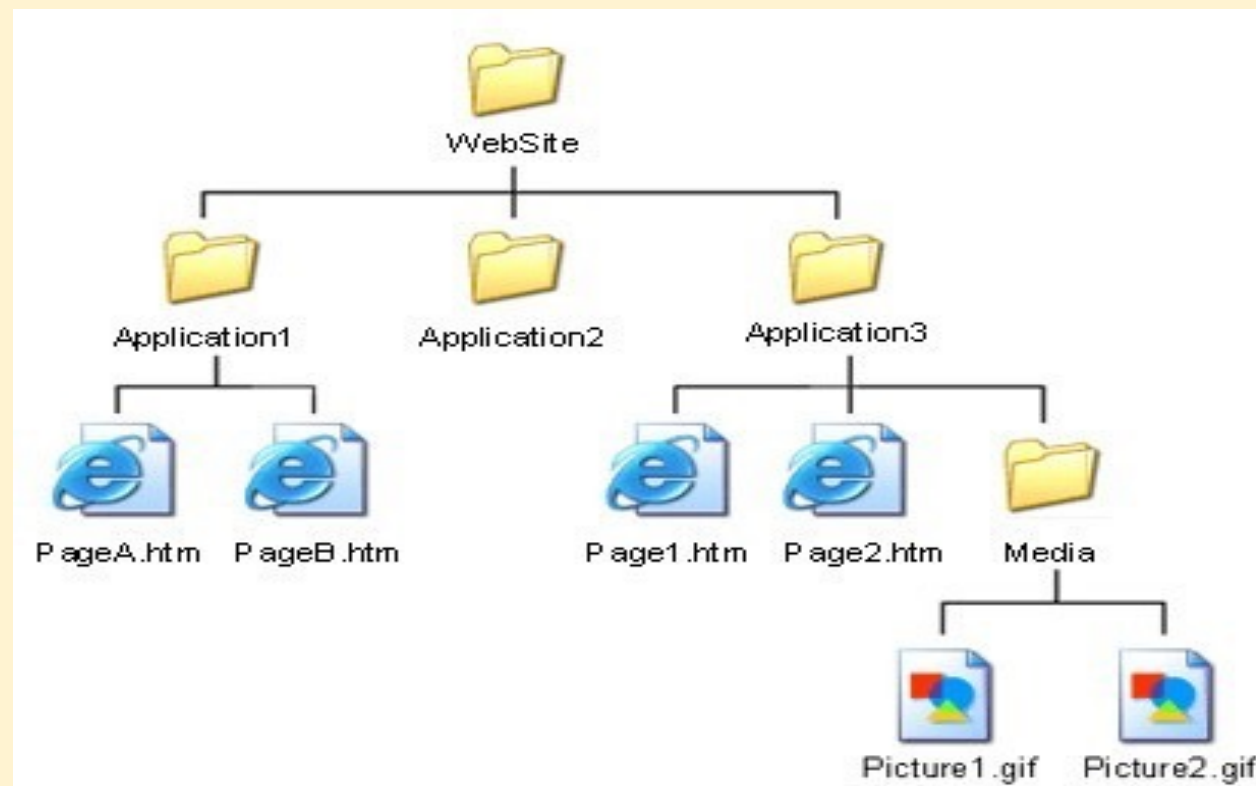




OS Services / Task of OS

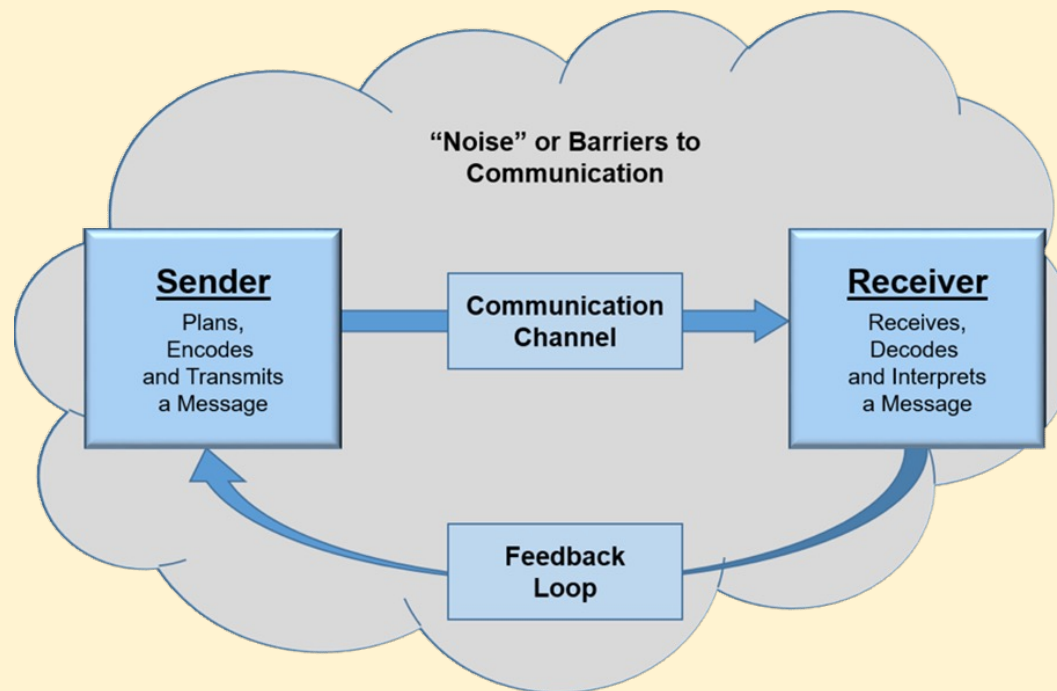
5. Controlled **access** to file

- In case of file access, OS provides a directory hierarchy for easy access and management of file.
- OS provides various file handling commands using which user can easily read, write and modify file.



6. Communication

- In multitasking environment, the processes need to communicate with each other and to exchange their information.
- Operating system performs the communication among various types of processes in the form of shared memory.





OS Services / Task of OS

7. Error detection and response

- An error may occur in CPU, in I/O devices or in the memory hardware.
- Following are the major activities of an operating system with respect to error handling –
 - The OS constantly checks for possible errors.
 - The OS takes an appropriate action to ensure correct and consistent computing.





OS Services / Task of OS

8. Accounting

- Keeping a track of which users are using how much and what kinds of computer resources can be used for accounting or simply for accumulating usage statistics.
- Usage statistics is used to reconfigure the system to improve computing services.





OS Services / Task of OS

9. Protection & Security

- Protection involves ensuring that all accesses to system resources is controlled.
- To make a system secure, the user needs to authenticate himself or herself to the system.





OS Services / Task of OS (Revision)

1. Program development
2. Program execution
3. Access to I/O devices
4. Memory management
5. Controlled access to file
6. Communication
7. Error detection and response
8. Accounting
9. Protection & Security



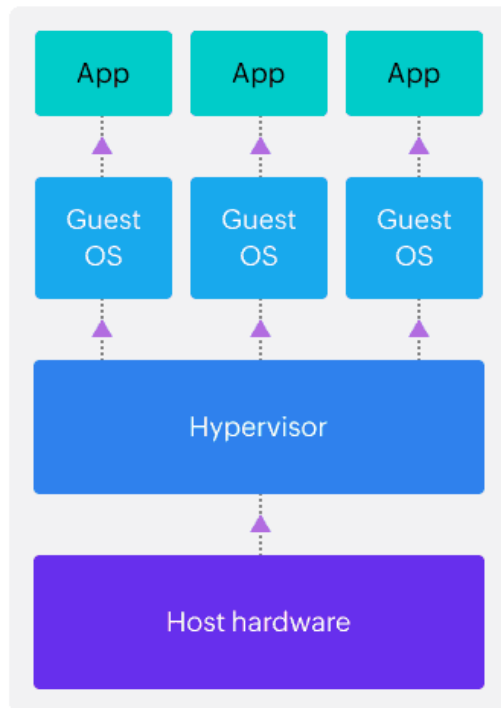
Virtualization

- The process of creating a virtual version of physical resources (hardware, storage, network, OS, etc.).
- Virtualization abstracts resources.
- Enables multiple systems to operate on a single hardware platform.

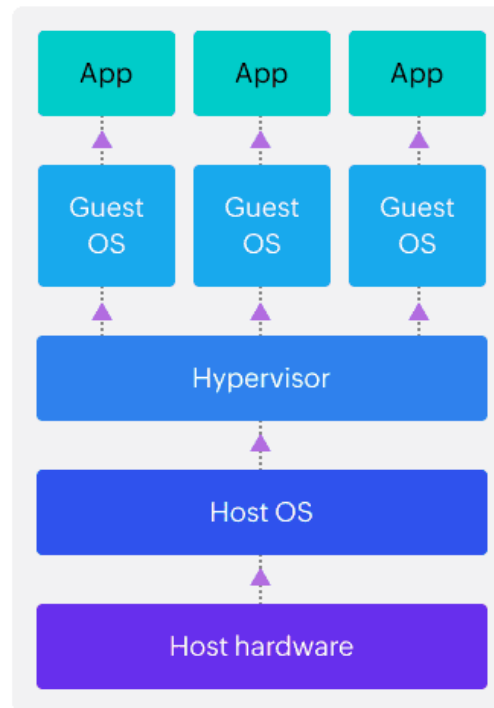


Virtualization

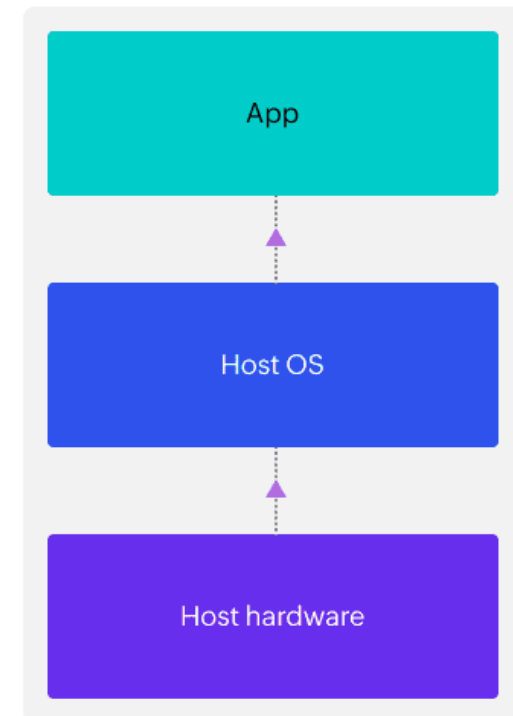
Virtualization architecture



Bare-metal hypervisor



Hosted hypervisor



Conventional server



Why Virtualization Matters

Benefits:

- Optimizes hardware usage.
- Reduces costs.
- Enables scalability and flexibility.
- Improves disaster recovery.

Real-World Use Cases:

- Cloud services.
- Application testing.
- Server consolidation.



Types of Virtualization

- **Hardware Virtualization:**

- Virtual Machines (e.g., VMware, VirtualBox)

- **Software Virtualization:**

- Virtual Operating Systems

- **Storage Virtualization:**

- Logical storage pools (e.g., RAID, SAN)

- **Network Virtualization:**

- Abstracting network resources (e.g., VLANs)



Virtual Machines and Hypervisors

•Virtual Machines (VMs):

- Definition: Software emulation of hardware.
- Example: Running Linux on a Windows system.

•Hypervisors:

- Type 1 (Bare-metal): Directly on hardware (e.g., VMware ESXi).
- Type 2 (Hosted): On an operating system (e.g., VirtualBox).



Real-World Applications

- **Cloud Computing:** AWS, Azure, GCP
- **Development Environments:** Testing isolated apps.
- **Server Consolidation:** Reducing hardware.
- **Disaster Recovery:** Backup and restore capabilities.



P P SAVANI
UNIVERSITY

Thank you 😊



P P SAVANI
UNIVERSITY

Additional Study Material



System calls

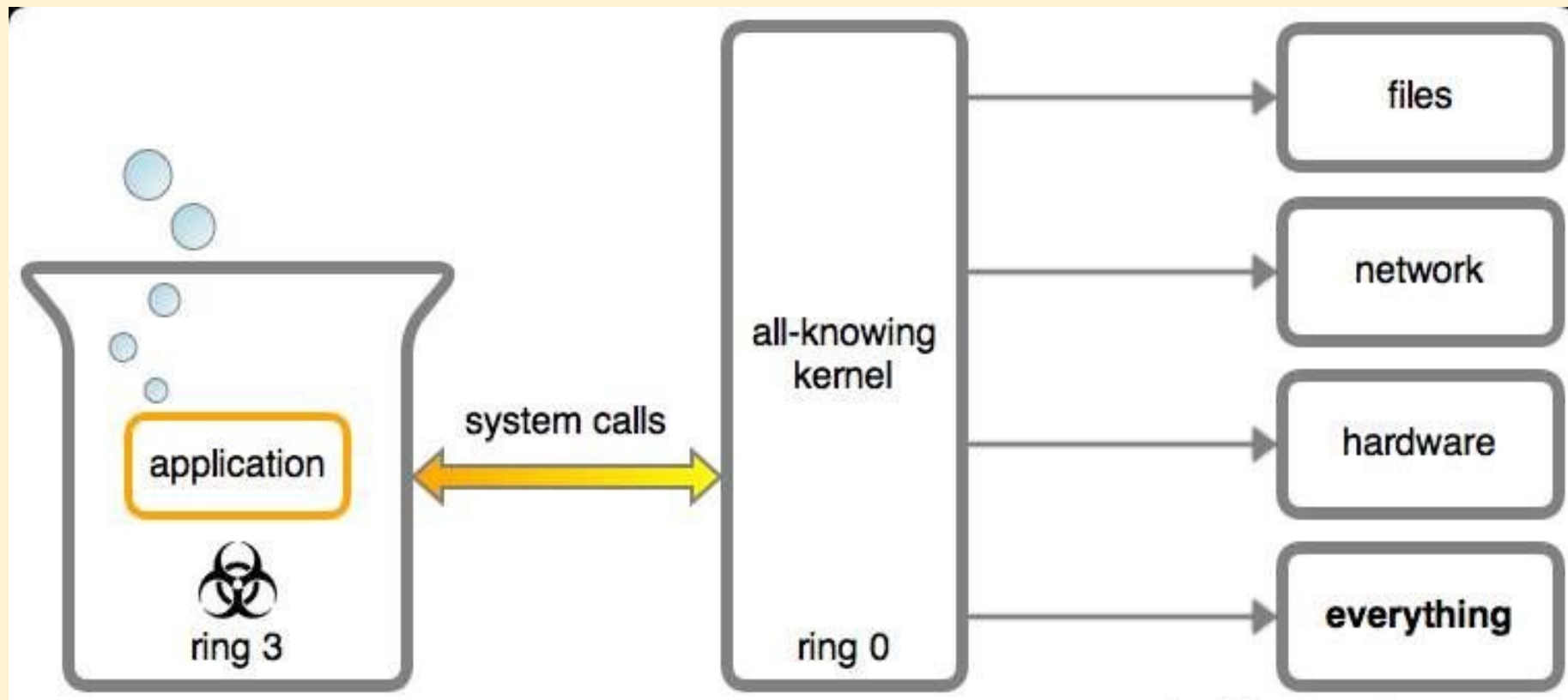
- The interface between OS and user program is defined by the set of system calls that OS provides.
- System calls vary from OS to OS.

Example: In Unix Read system call is

- `count=read(fd, buffer, nbytes)`
 - fd is a file descriptor.
 - When a file is opened, permissions are checked.
 - If access is allowed, a number (fd) is returned.
 - Then file can be read/written.
 - nbytes is number of bytes to read
 - buffer is where read deposits (stores) the data

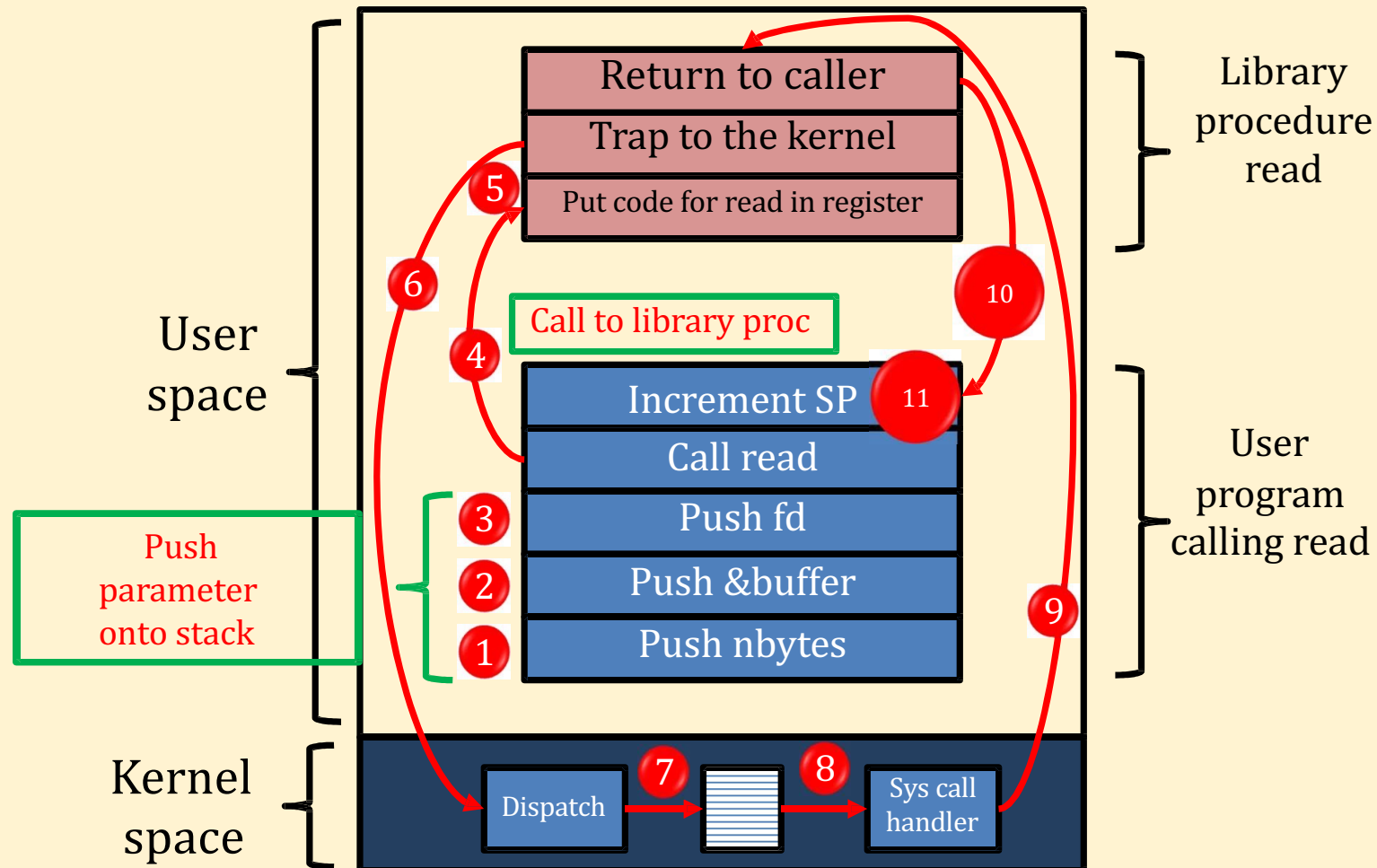


System calls





System calls (Cont...)





System Calls (Cont...)

<u>Director and file system management</u>	
Call	Description
<code>s = mkdir(name,mode)</code>	Create a new directory
<code>s = rmdir(name)</code>	Remove an empty directory
<code>s = link(name1, name2)</code>	Create a new entry, name2, pointing to name1
<code>s = unlink(name)</code>	Remove a directory entry
<code>s = mount(special, name, flag)</code>	Mount a file system
<code>s = umount(special)</code>	Unmount a file system
<u>Miscellaneous</u>	
Call	Description
<code>s = chdir(dir name)</code>	Change the working directory
<code>s = chmod(name,mode)</code>	Change a file's protection bits
<code>s = kill(pid, signal)</code>	Send a signal to a process
<code>seconds = time(&seconds)</code>	Get the elapsed time since Jan. 1, 1970



System Calls (Cont...)

<u>Process management</u>	
Call	Description
<code>pid = fork()</code>	Create a child process identical to the parent
<code>pid = waitpid(pid, &statloc, options)</code>	Wait for a child to terminate
<code>s = execve(name, argv, environp)</code>	Replace a process' core image
<code>exit(status)</code>	Terminate process execution and return status
<u>File management</u>	
Call	Description
<code>fd = open(file, how, ...)</code>	Open a file for reading, writing, or both
<code>s = close(fd)</code>	Close an open file
<code>n = read(fd, buffer, nbytes)</code>	Read data from a file into a buffer
<code>n = write(fd, buffer, nbytes)</code>	Write data from a buffer into a file
<code>position = lseek(fd, offset, whence)</code>	Move the file pointer
<code>s = stat(name, &buf)</code>	Get a file's status information



Roles of OS (OS can be viewed as)

1. The OS as an Extended/Virtual Machine
2. The OS as a Resource Manager



OS as Resource Manager

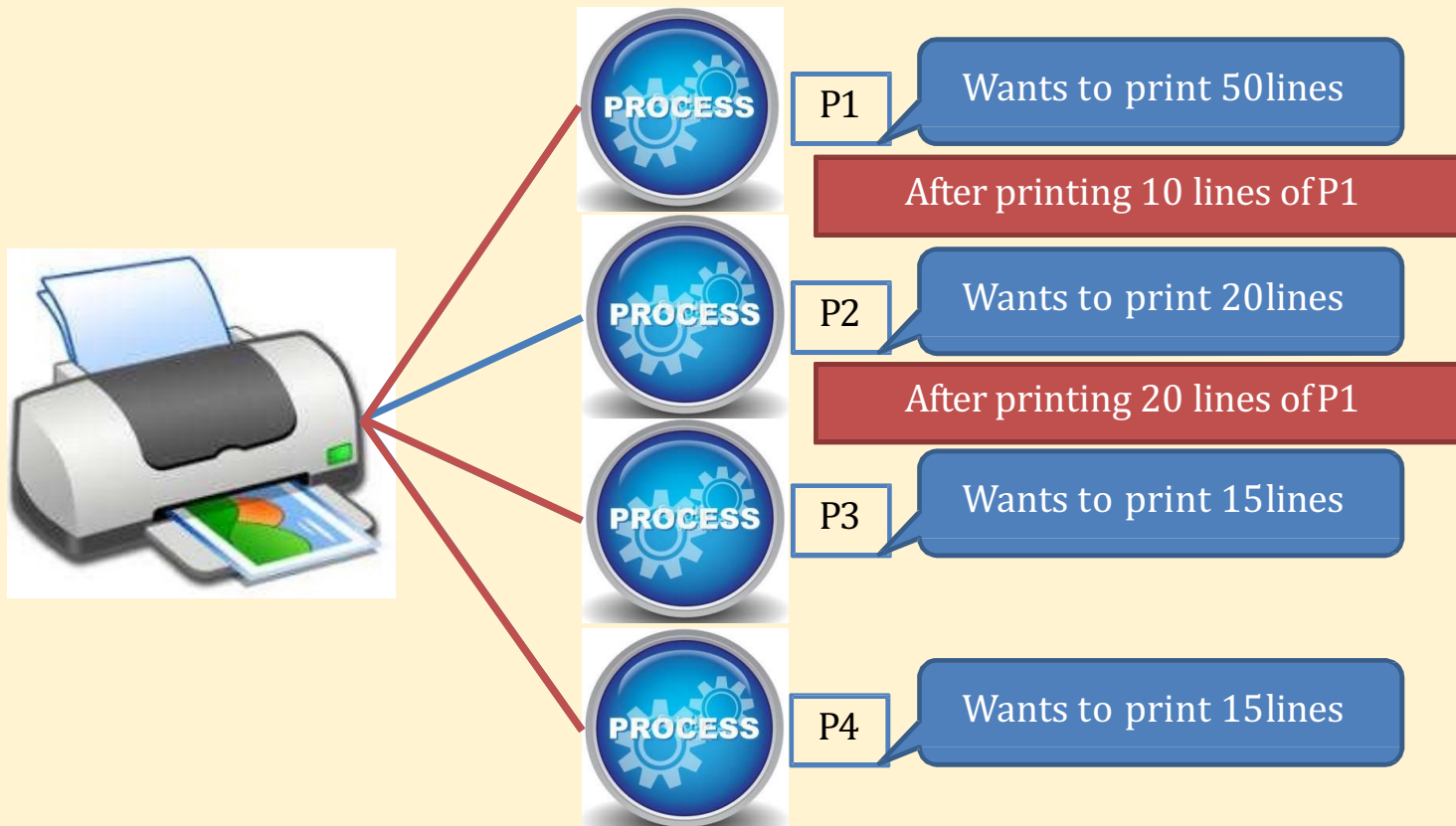
- ❖ There are lots of resources in computer system
 - CPU (Processor)
 - Memory
 - I/O devices such as hard disk, mouse, keyboard, printer, scanner etc.
- ❖ If a computer system is used by multiple applications (or users), then they will compete for these resources.





OS as Resource Manager

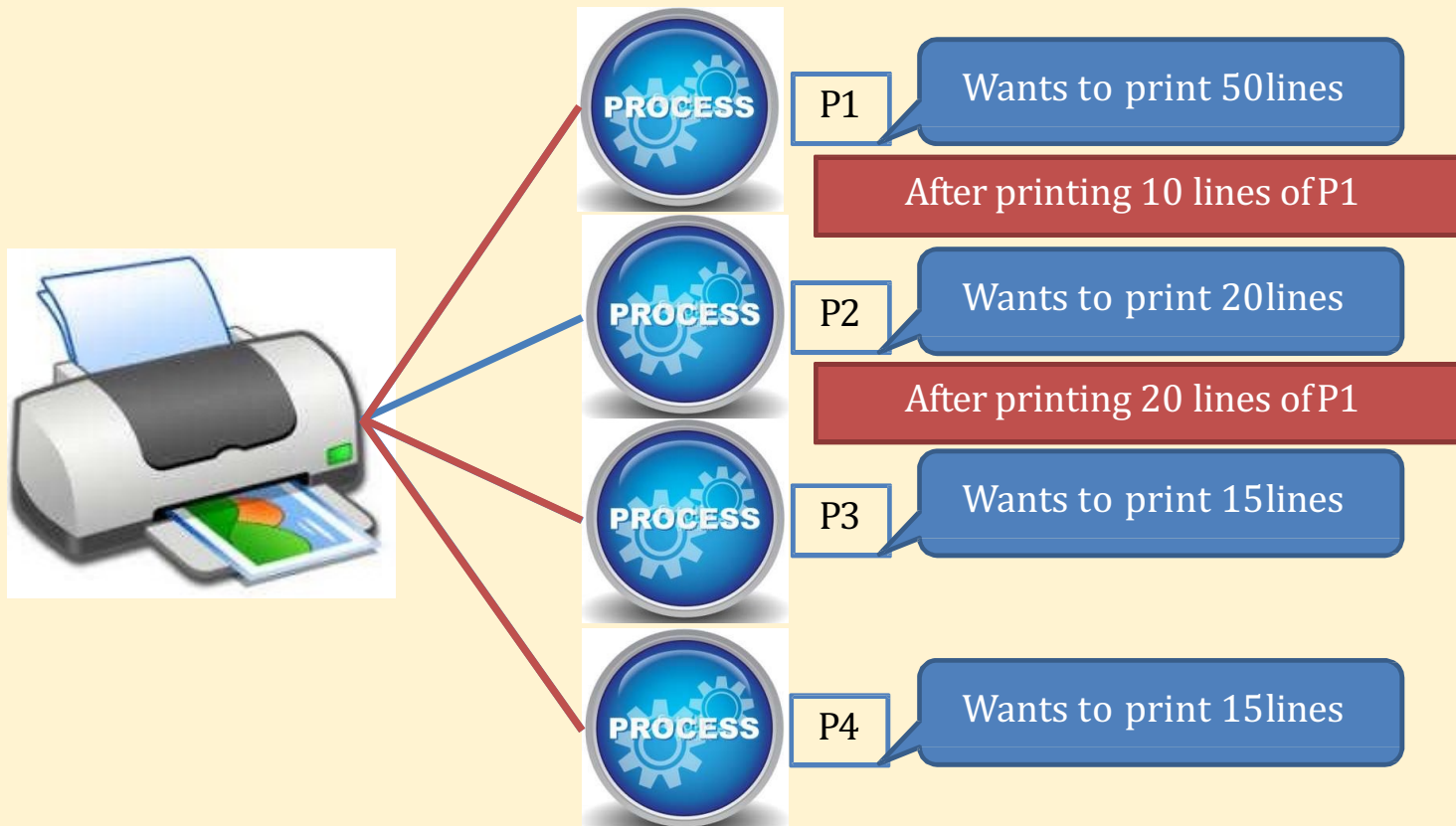
- ❖ It is the job of OS to allocate these resources to the various applications so that:
 - The resources are allocated fairly (equally)





OS as Resource Manager

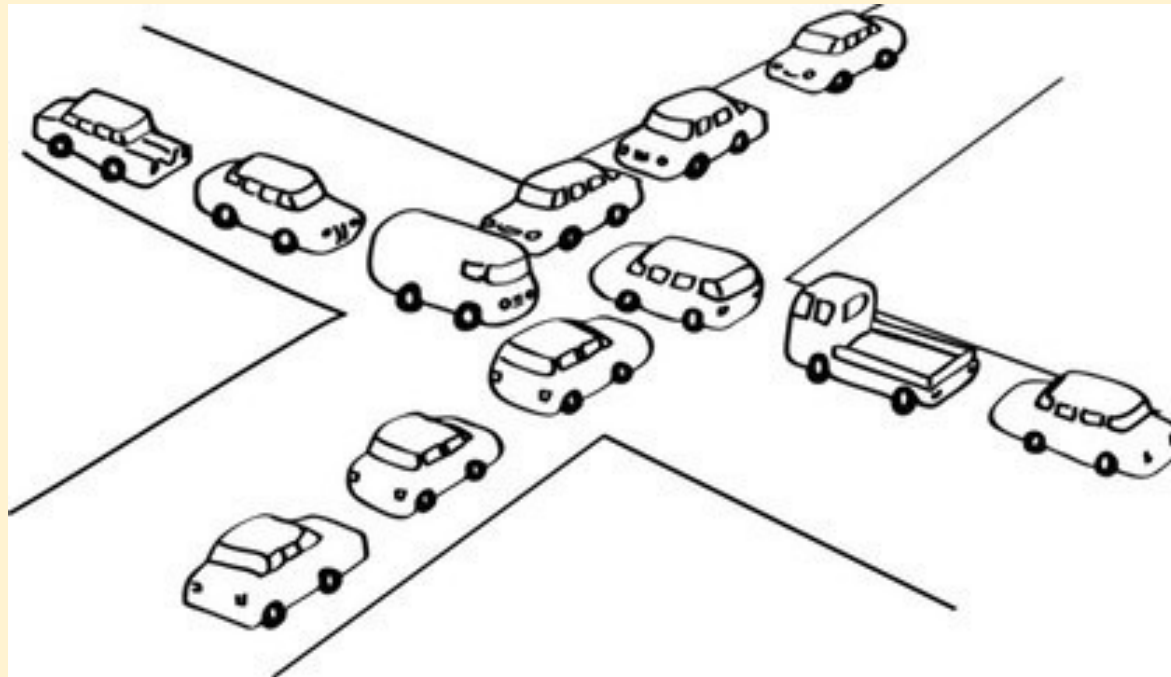
- ❖ It is the job of OS to allocate these resources to the various applications so that:
 - The resources are allocated protected from cross-access





OS as Resource Manager

- ❖ It is the job of OS to allocate these resources to the various applications so that:
 - Deadlock are detected, resolved and avoided.

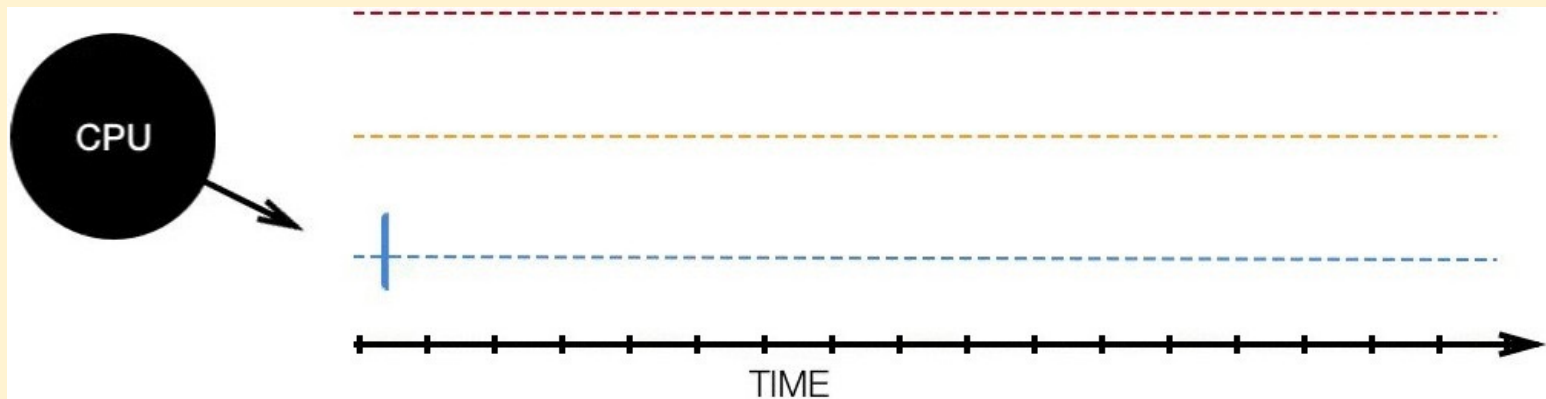


OS as Resource Manager (Cont...)



P P SAVANI
UNIVERSITY

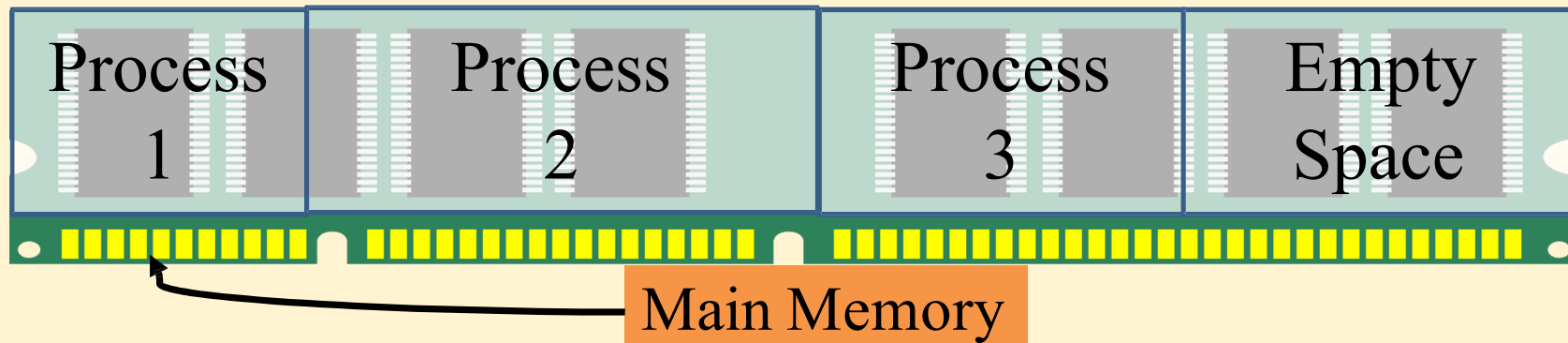
- ❖ Resource manager – sharing resources in two different ways:
 1. In time sharing/multiplexing (i.e CPU)





OS as Resource Manager (Cont...)

- ❖ Resource manager – sharing resources in two different ways
 1. In time sharing/multiplexing. (i.e CPU)
 2. In space sharing/multiplexing. (i.e Memory)





Basic resources of computer system

- ❖ Following are the resources of computer system, managed and allocated by Operating System:
 - CPU (Central Processing Unit)/ Process Management
 - Memory Management
 - Primary Memory
 - Secondary Memory
 - File system
 - I/o Device

Process Management

OS as Resource Allocator



P P SAVANI
UNIVERSITY

- A *process* is a program in execution. A process needs certain resources, including CPU time, memory, files, and I/O devices, to accomplish its task.
- The operating system is responsible for the following activities in connection with process management.
 - Process creation and deletion.
 - process suspension and resumption.
 - Provision of mechanisms for:
 - process synchronization
 - process communication



Main Memory Management

OS as resource allocator

- Memory is a large array of words or bytes, each with its own address. It is a repository of quickly accessible data shared by the CPU and I/O devices.
- Main memory is a volatile storage device. It loses its contents in the case of system failure.
- The operating system is responsible for the following activities in connections with memory management:
 - **Keep track of which parts of memory are currently being used and by whom.**
 - **Decide which processes to load when memory space becomes available.**
 - **Allocate and deallocate memory space as needed.**

Secondary Storage management

OS as Resource allocator



P P SAVANI
UNIVERSITY

- Since main memory (*primary storage*) is volatile and too small to accommodate all data and programs permanently, the computer system must provide *secondary storage* to back up main memory.
- Most modern computer systems use disks as the principle on-line storage medium, for both programs and data.
- The operating system is responsible for the following activities in connection with disk management:
 - **Free space management**
 - **Storage allocation**
 - **Disk scheduling**

I/O System Management

OS as resource allocator



P P SAVANI
UNIVERSITY

The I/O system consists of:

- A buffer-caching system
- A general device-driver interface
- Drivers for specific hardware devices



File Management

OS as resource allocator

- A file is a collection of related information defined by its creator. Commonly, files represent programs (both source and object forms) and data.
- The operating system is responsible for the following activities in connections with file management:
 - File creation and deletion.
 - Directory creation and deletion.
 - Support of primitives for manipulating files and directories.
 - Mapping files onto secondary storage.
 - File backup on stable (nonvolatile) storage media.



Protection System

OS as control program

- *Protection* refers to a mechanism for controlling access by programs, processes, or users to both system and user resources.
- The protection mechanism must:
 - distinguish between authorized and unauthorized usage.
 - specify the controls to be imposed.
 - provide a means of enforcement.



OS as Extended Machine

The architecture of a computer is difficult to program

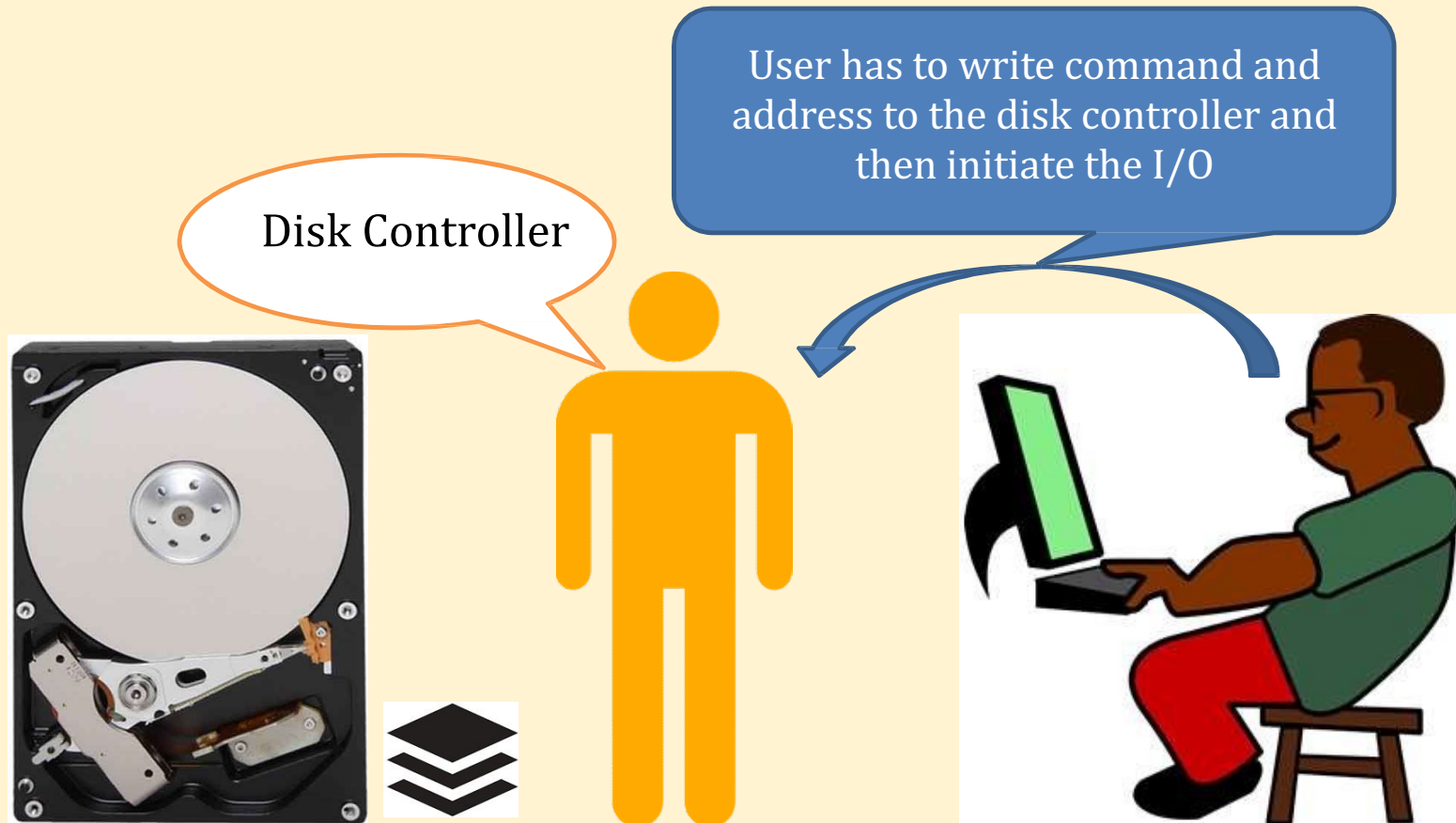
- Architecture (instruction set, memory organization, I/O, bus structure) of most of computer at the machine level language is primitive and awkward to program.
- Example: If user want to read from floppy or hard disk:





OS as Extended Machine

- Example: If user want to read from floppy or hard disk:





OS as Extended Machine

- Example: If user want to read from floppy or hard disk:

The disk controller will find the requested data in the disk and fetch it from disk to disk controller buffer.

User has to check the status of disk controller operation where it has finished or not.





OS as Extended Machine

- Example: If user want to read from floppy or hard disk:

If success, the data from disk controller buffer should be moved to main memory (to the application buffer).





OS as Extended Machine (Cont...)

- If all the users will have to do these messy details:
 - The program will be very difficult to write and quite long.
 - The program will be hardware dependent.
- User don't want to be involved in programming of storage devices.
- Therefore, an OS provides a set of basic commands or instructions to perform various operations such as read, write, modify, save or close.
- Dealing with these command is easier than directly dealing with hardware.
- Operating system hides the complexity of hardware and present a beautiful interface to the users.



Operating system structures

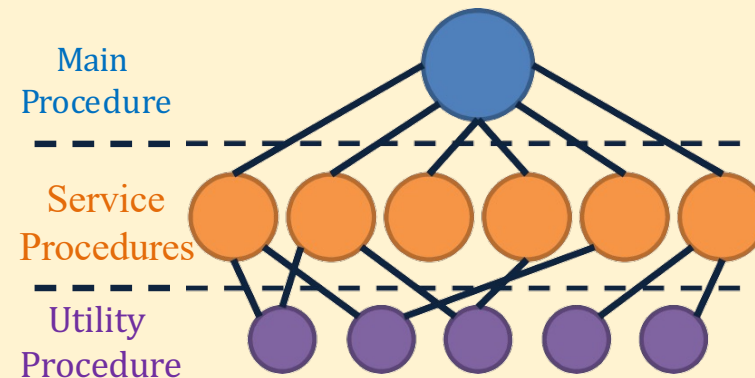
1. Monolithic systems
2. Layered systems
3. Microkernel
4. Client-server model
5. Virtual machines
 - VM/370
 - Virtual machines rediscovered
 - The java virtual machine
6. Exokernels



Monolithic systems

- The entire OS runs as a single program in kernel mode.
- OS is written as a collection of procedures, linked together into a single large executable binary program.
- Each procedure has well defined interface in terms of parameter and results, and each one is free to call any other one.

- A main program that invoke the requested service procedure.
- A set of service procedures that carry out the system calls.
- A set of utility procedures that help the service procedure.





Layered systems

- In this system, the OS is organized as a hierarchy of layers.
- The first system constructed in this way was THE system.



Architecture of THE system

Layer	Function	Description
5	The operator	Operator was located.
4	User Programs	User programs were found.
3	Input / Output Management	Takes care of managing the I/O devices. Buffering the information.
2	Operator- process communication	Handles communication between each process and the operator console (i.e. user).
1	Memory and drum managemen t	Did the memory management. Allocated space for process in main memory and on a 512K word drum used for holding parts of processes for which there was no room in memory.
0	Processor allocation and multi- programmin g	Provided the basic multiprogramming of the CPU. Dealt with allocation of the processor, switching between processes when interrupts occurred or timers expired.

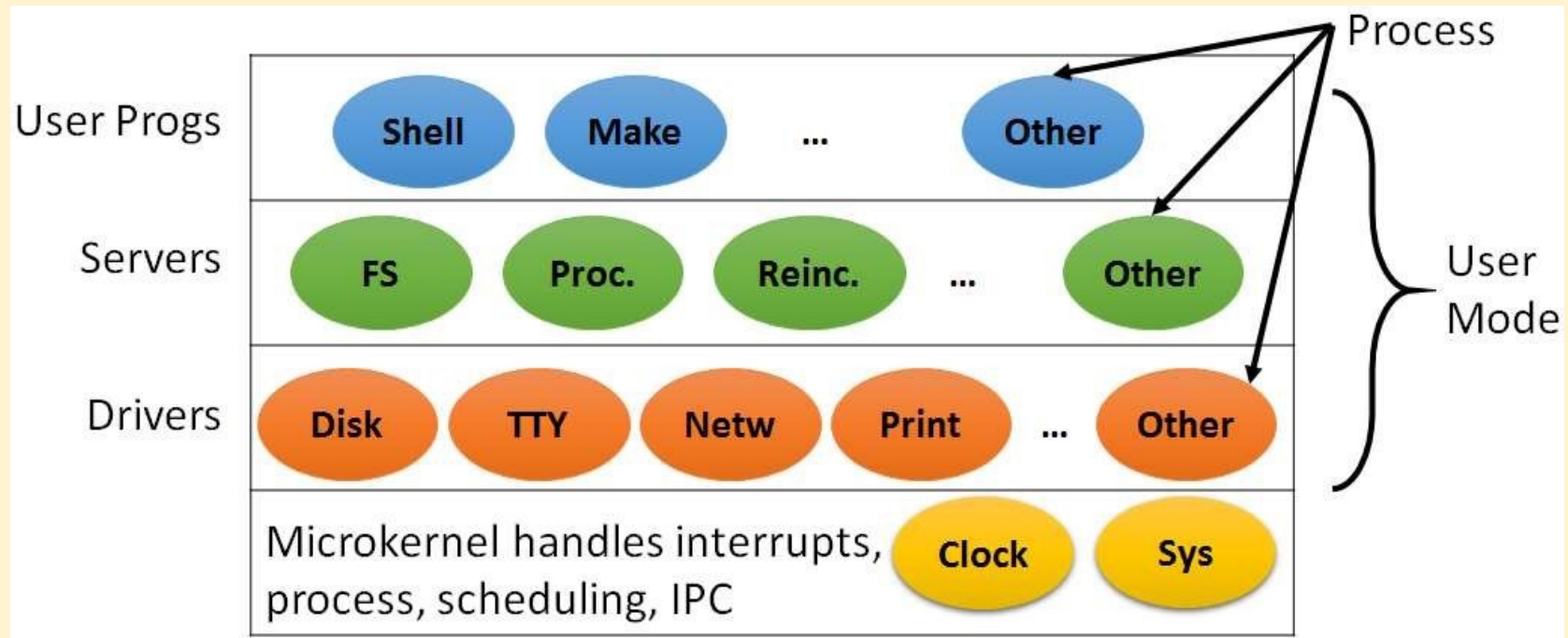


Microkernel

- In layered approach, the designer have choice where to draw the kernel and user mode boundary.
- It is better to put as little as possible in kernel mode because bugs in the kernel can bring down the system instantly.
- The microkernel design provides high reliability by splitting OS up into small well defined modules, only one module run in kernel and rest of all run in user mode.
- As each device driver runs as a user process, a bug in audio driver will cause the sound to be stop, but not crash the computer.
- Examples: Integrity, K42, QNX, Symbian and MINIX 3.



Microkernel (Cont...)

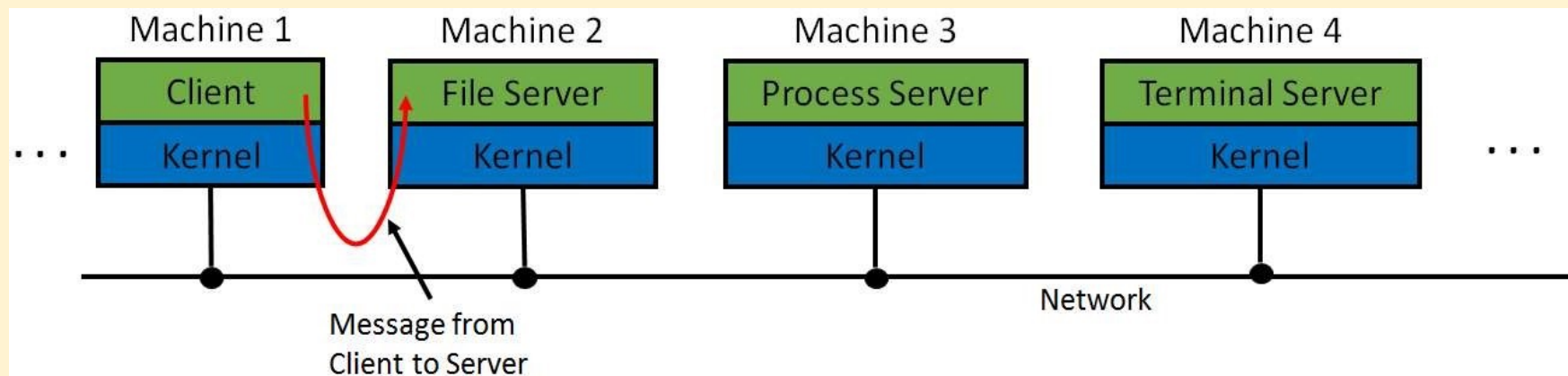


- Kernel contains only
 - Sys (Kernel call handler)
 - Clock (because scheduler interact with it)



Client-Server model

- Processes are divided into two categories
 - Servers: provide services
 - Clients: use services
- Client and server run on different computers, connected by LAN or WAN and communicate via **message passing**.
- To obtain a service, a client constructs a message saying what it wants and sends it to the server.
- The server then does the work and sends back the answer.



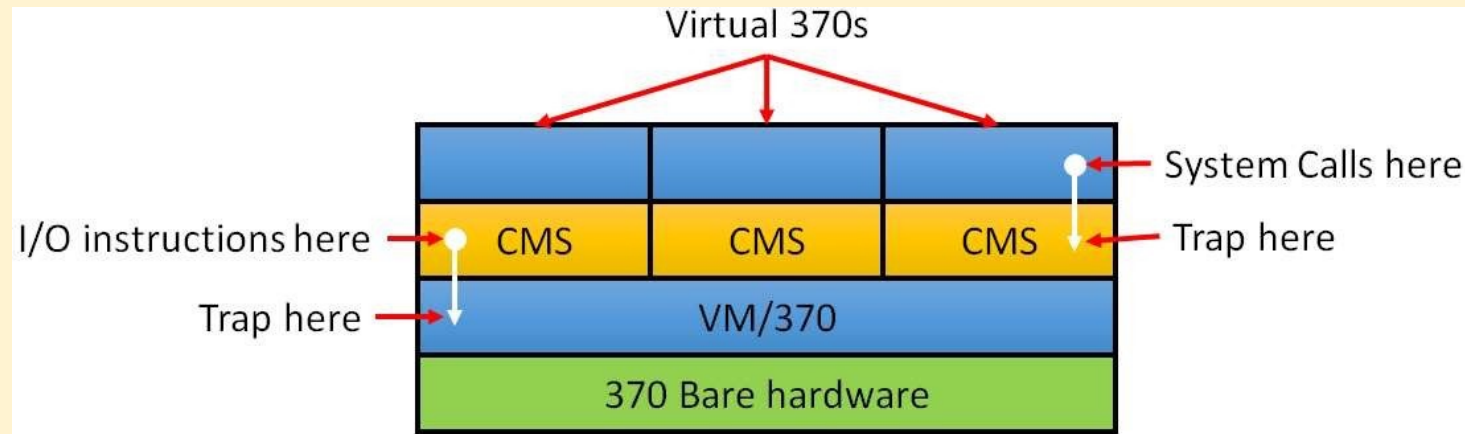


Virtual machines

- The initial releases of OS/360 were strictly batch systems.
- But many users wanted to be able to work interactively at a terminal, so OS designers decided to write timesharing systems for it.
- Types of Virtual machines are:
 - VM/370
 - Virtual Machines Rediscovered
 - The Java Virtual Machine



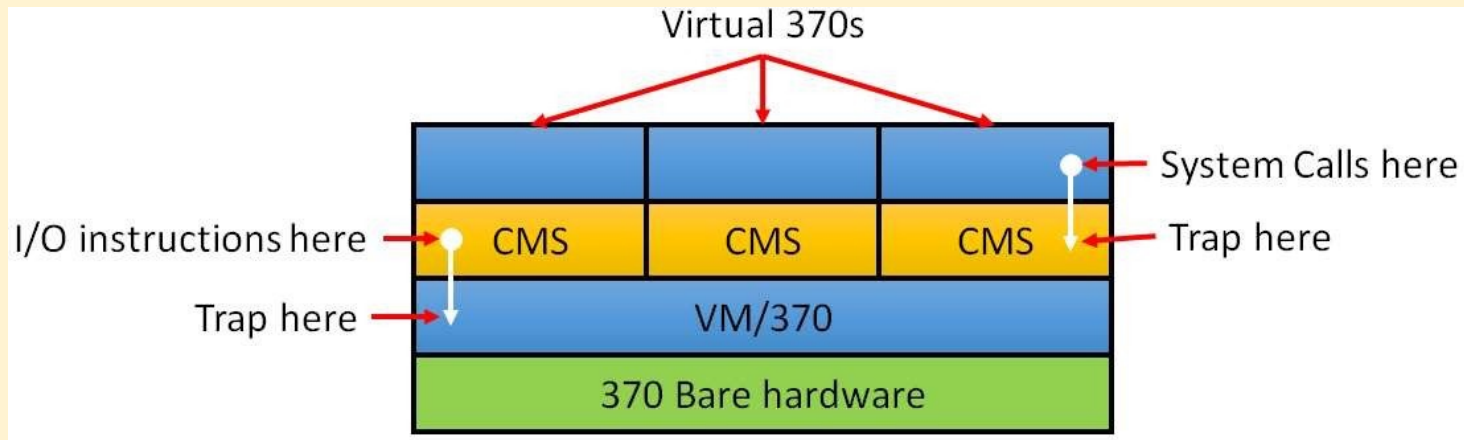
VM/370



- Virtual machine monitor run on the bare hardware and does the multiprogramming.
- Each virtual machine is identical to the true hardware; each one can run any OS (may be different) that will run directly on the bare hardware.
- On VM/370, some run OS/360 while the others run single user interactive system called CMS (Conversational Monitor System) for interactive time sharing users.



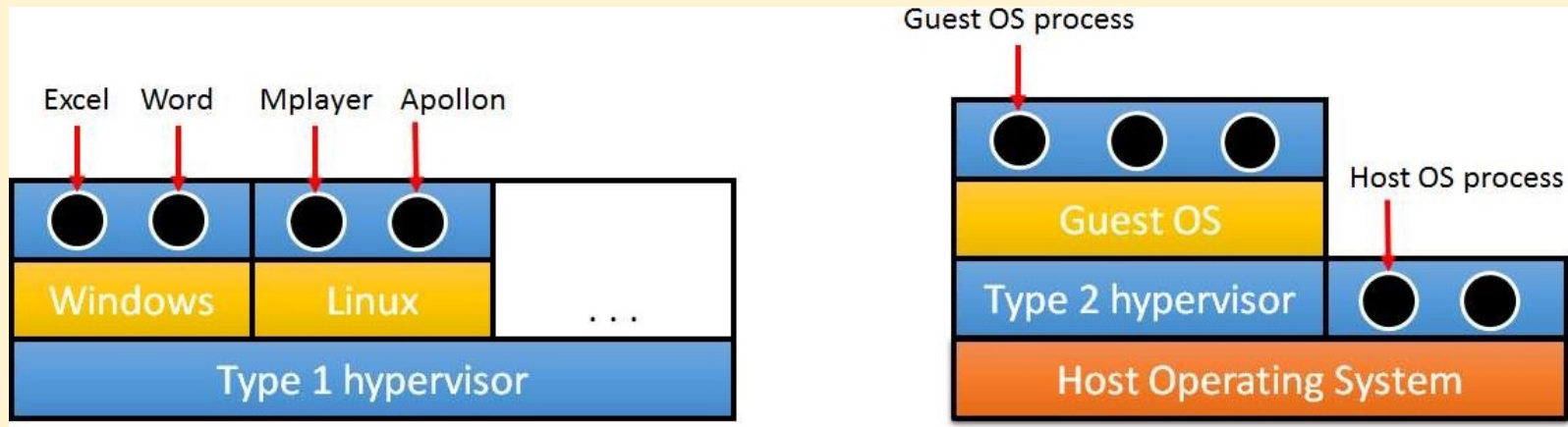
VM/370 (Cont...)



- When CMS program executed a system call, a call was trapped to the operating system in its own virtual machine, not on VM/370.
- CMS then issued the normal hardware I/O instruction for reading its virtual disk or whatever was needed to carry out the call.
- These I/O instructions were trapped by VM/370 which then performs them.



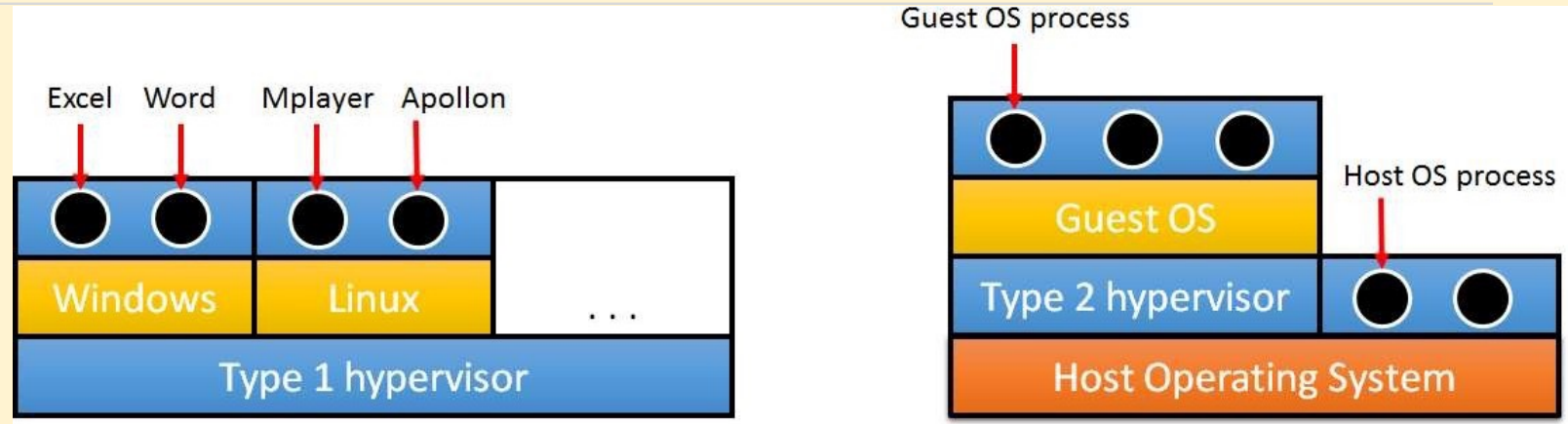
Virtual Machines Rediscovered



- Companies can run their mail servers, web servers, FTP servers and other servers on the same machine without having a crash of one server bring down the rest.
- Web hosting company offers virtual machines for rent, where a single physical machine can run many virtual machines; each one appears to be a complete machine.
- Customers who rent a virtual machine can run any OS or software.



Virtual Machines Rediscovered (Cont...)

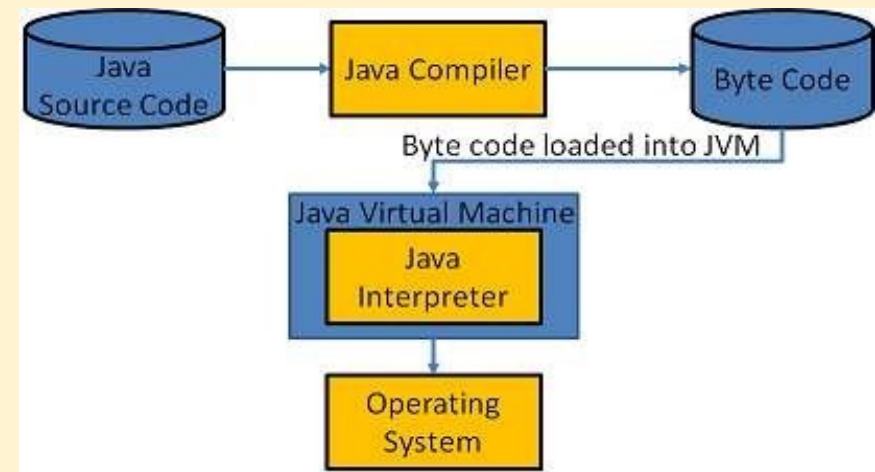


- Another use of virtualization is for end users who want to be able to run two or more operating systems at the same time, say Windows and Linux.
- Type 1 hypervisors runs on the bare hardware.
- Type 2 hypervisors run as application programs on top of Windows, Linux, or some other operating system, known as the host operating system.



Java Virtual Machine

- When Sun Microsystems invented the Java programming language, it also invented a virtual machine called the JVM (Java Virtual Machine).
- The Java compiler produces code for JVM, which then typically is executed by a software JVM interpreter.
- The advantage is that the JVM code can be shipped over the internet to any computer that has a JVM interpreter and run there.





Exokernels

- Rather than cloning (copying) the actual machine, another strategy is partitioning it (giving each user a subset of the resource).
- For example one virtual machine might get disk blocks 0 to 1023, the next one might get block 1024 to 2047, and so on.
- Program running at the bottom layer (kernel mode) is called the exokernel.
- Its job is to allocate resources to virtual machines and then continuously check to make sure no machine is trying to use somebody else's resources.
- The advantage of the exokernel scheme is that it saves a layer of mapping.



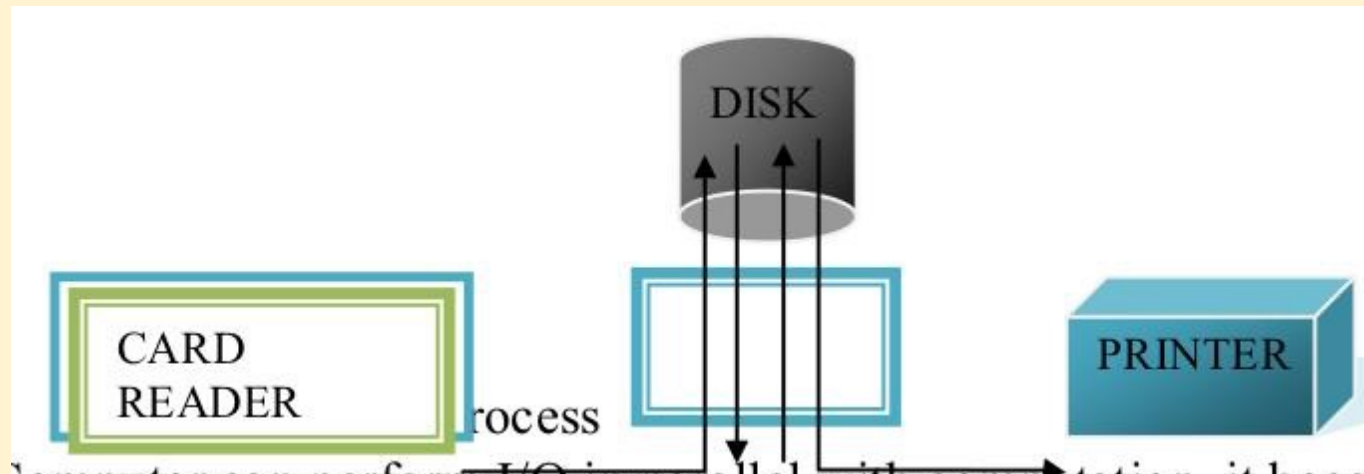
Spooling

- It allows the CPU to Overlap I/O of one job with computation of other jobs.
- While executing one job, the OS.
 - Reads next job from card reader into a storage area on the disk (job queue).
 - Outputs printout of previous job from disk to printer.
 - This process is called **spooling**.
- Spooling refers to putting jobs in a buffer, a special area in memory or on a disk where a device can access them when it is ready.
- Spooling is useful because device access data with different rates. The buffer provides a waiting station where data can rest while the slower device catches up.



Spooling

- The most common spooling application is print spooling.
 - In print spooling, documents are loaded into a buffer and then the printer pulls them off from the buffer at its own rate.
 - Each applications' output is spooled to a separate disk file.
 - Copies the queued spool files to the printer one at a time.
 - Spooling overlaps the I/O of one job with the computation of other jobs.





Spooling

- *Job pool* – data structure that allows the OS to select which job to run next in order to increase CPU utilization.
- Spooling is also used for processing data at remote sites.
 - The CPU sends the data via communications path to a remote printer.
- Spooling batch systems were the first and are the simplest of the multiprogramming systems.